Software Analysis: Dynamic Views of the LavaRápido System

Juan Camilo Guilombo Osorio, Brayan Estiven Patiño Cabrera and Merari Urbano Quintero

Centro de la Industria, la Empresa y los Servicios, Servicio Nacional de Aprendizaje (SENA), Huila Regional

Software Analysis and Development Technologist program (ADSO), group (ficha) 3145556

Instructor: Karol Daniela Correa

Version 2.8, October 27, 2026

Introduction

This document shows how LavaRápido behaves when someone uses it: what happens, in which order, and between which screens and services. It is organized in four parts:

1. **Use cases by actor**, with their «include»/«extend» relationships.
2. **Sequence diagrams** of the main flows: from the screen to the gateway, from the gateway to the service, and from the service to the database or to another service.
3. **Activity diagrams** of the business processes.
4. **State diagrams** of the entities that change status (booking, payment, service execution, promotion and inspection report).

On October 21, 2026, the following were updated against the code of that day: the sign-out that voids the token (section 2.8), the lock on operator assignment (section 2.4), the receipt of at most 2 MB and the cash option that only appears while its account is active (sections 2.5 and 3.4), and the handling of bookings that ended without anyone closing them (sections 4.1 and 4.3): the system only changes the status when there is no doubt, and the rest goes to the administrator's "To close" list. On October 20, 2026, the content was checked again against the code: the client only cancels (the administrator reschedules), the role change of accounts was removed, the sequences and states that did not match were corrected, and the use case and other diagrams were redrawn with straight lines and no crossings. On October 19, 2026, use cases CU31 and CU32, password recovery (section 2.10) and the account life cycle (section 4.6) were added, and the screenshots of the screen inventory were taken again with demo data. On October 17, 2026, the consent to the legal documents at registration (section 2.1) and the inspection report from the mobile app (section 2.9) were added. The diagrams are written in Mermaid. They were checked against the code and the running system on October 9, 2026, and updated against the code on October 15, 2026 with the account lockout, automatic assignment, operator choice, the service start window, the alert for bookings without an operator and the inspection report. The screen inventory, with screenshots by role, is at the end. On October 23, 2026 (version 2.5) it was added that recovering the password revokes all sessions (section 2.10) and that, if security-

service does not answer, the gateway lets the request through (*fail-open*); the rest of the analysis still holds against *develop* of that day. On October 24, 2026 (version 2.6), against *develop* of that day, the permission matrix became read-only (sections 2.7 and 3.6 and the screen inventory) and the automatic cancellation of expired scheduled bookings was removed, because every booking is created confirmed (section 4.1). On October 26, 2026 (version 2.7), against *develop* of that day, payment at booking was added: the customer chooses how they will pay when booking, with at most one active cash booking (sections 2.2 and 3.2); payment-service announces with `payment.reported` when the customer uploads the receipt (section 2.5), and a booking with digital payment that reaches its deadline without a receipt is cancelled automatically with the reason `PAYMENT_TIMEOUT`, after asking payment-service (sections 2.11 and 4.1). On October 27, 2026 (version 2.8), against `main` of that day (booking *v1.1.1*), a null, empty, or longer-than-10 list of services returns 400 `VALIDATION_ERROR` instead of 500, and the alternative times no longer include those that clash with another booking of the same vehicle (sections 2.2 and 3.2). The customer screenshots of the screen inventory (section 5.3) are those of October 19 and do not show these changes.

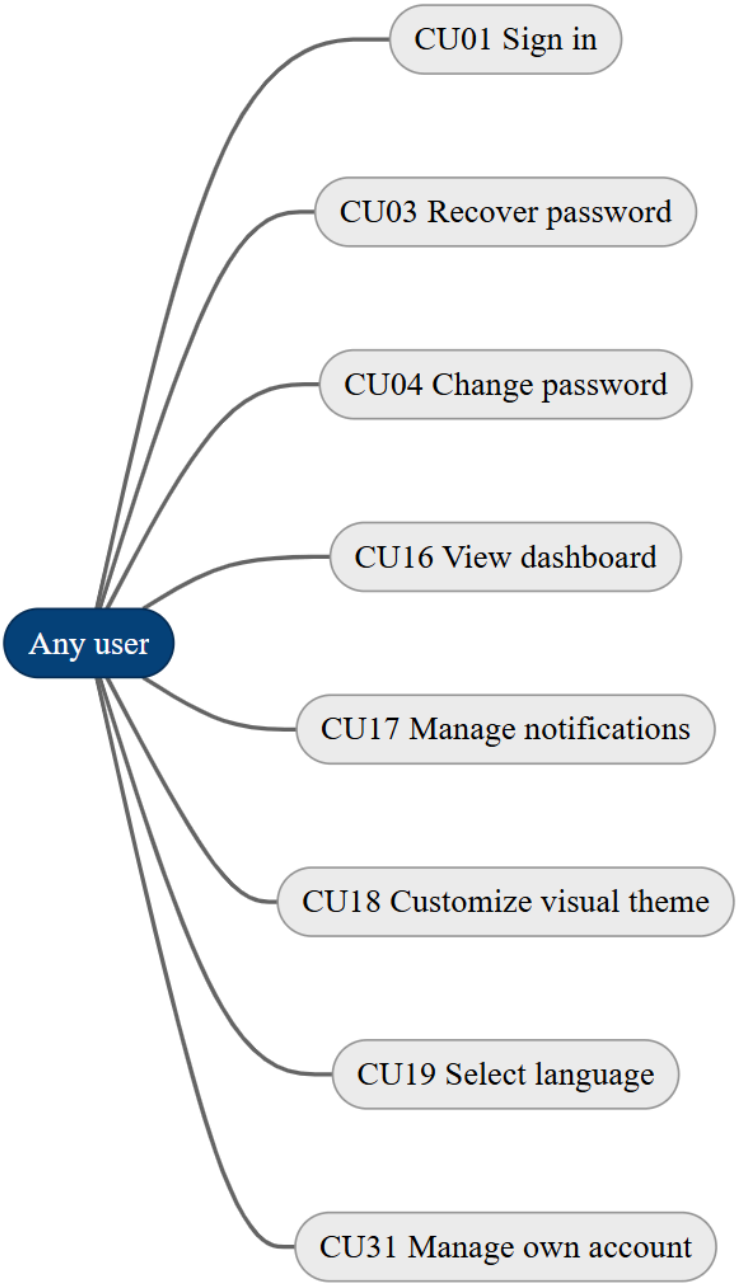
The full specification of each requirement and use case (actors, precondition, main flow, alternative flows and postcondition) is in the *Software Requirements Specification*, sections 5 and 7.3; here they are cited by their identifier (RF-XXX, CUXX).

1. Use Cases by Actor

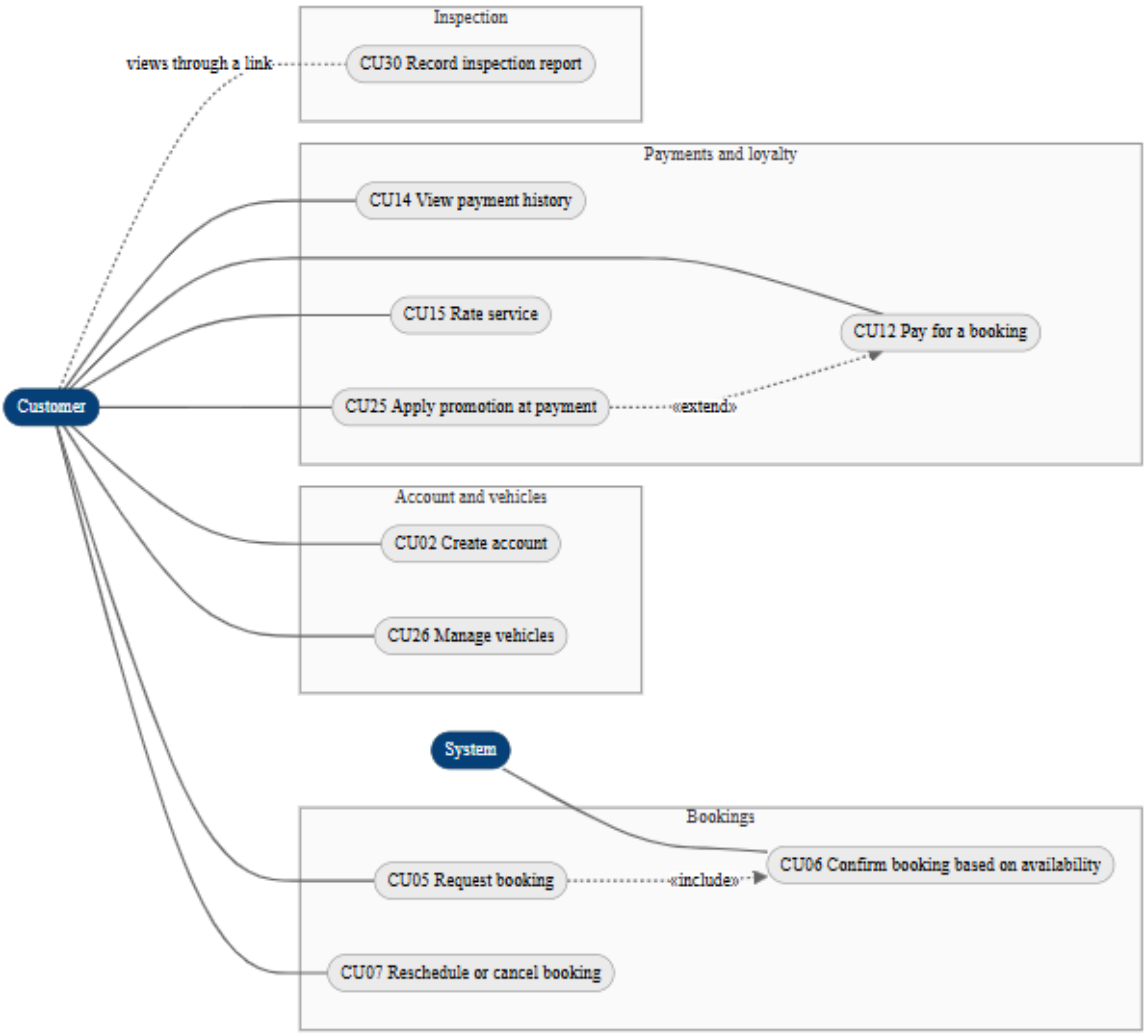
1.1 Use Case Diagram

The 32 use cases are presented in four diagrams, one per actor, grouped by module.

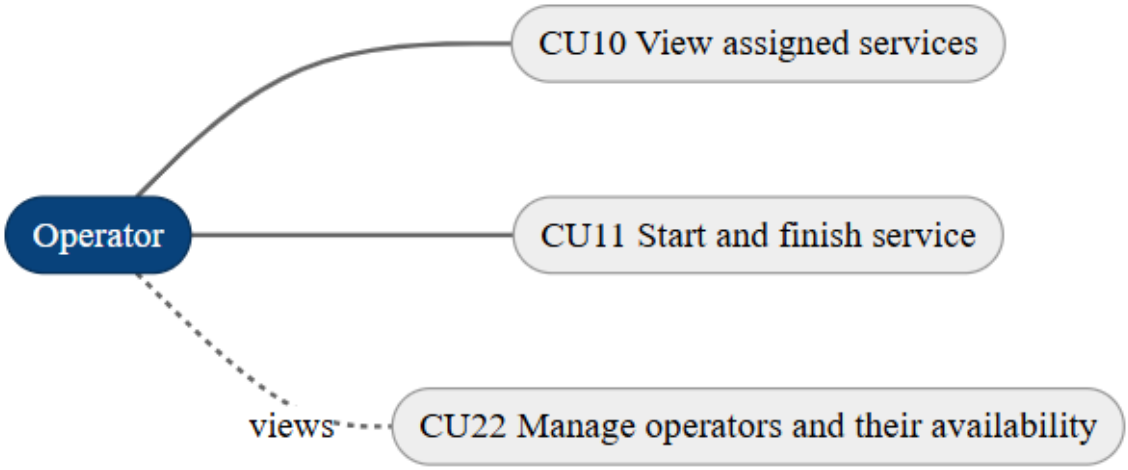
Use cases common to the three roles



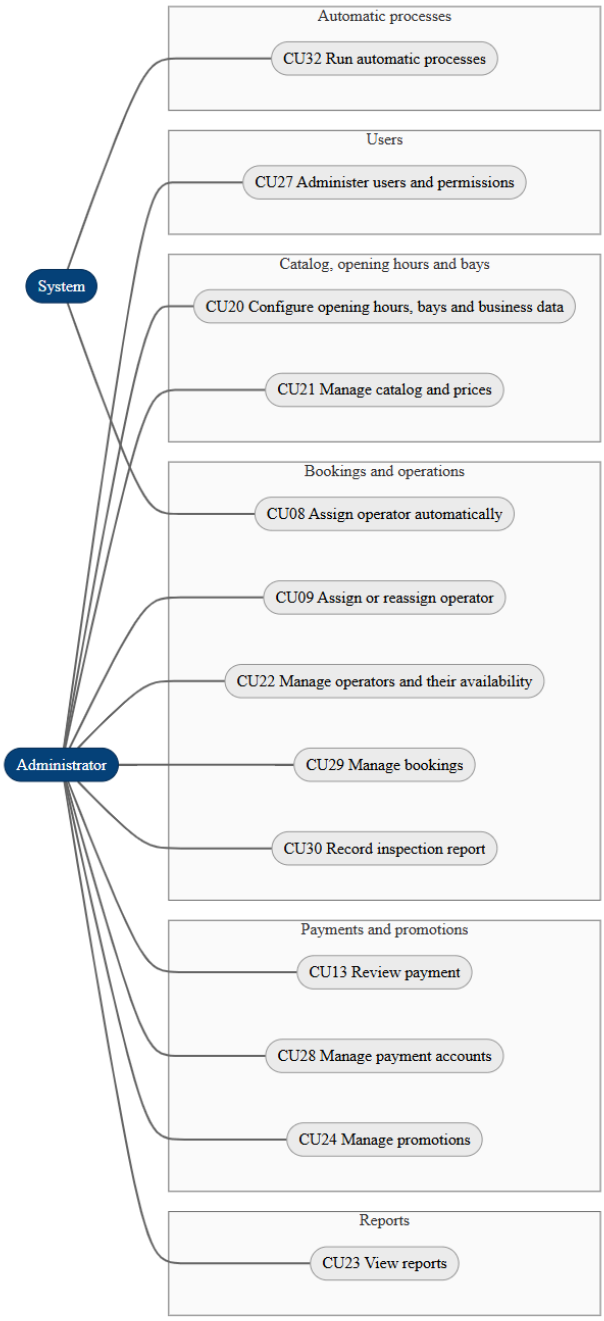
Customer use cases



Operator use cases



Administrator and System use cases



Note. «include» means that CU05 always runs CU06; «extend» means that CU25 optionally extends CU12. CU22 appears as view-only for the operator and as management for the administrator, who is the one recording shifts, absences and the operator profile (RF-022, RF-038). The system runs CU08 each time a booking is confirmed, and CU32 groups the processes

that run without anyone asking for them (reminders, alerts, notices and the first administrator). The customer does not record the inspection report (CU30): they view it through the link the administrator shares. In CU07 the customer cancels from the history or the payment screen; a new date or time is set by the administrator from Bookings (CU29), since neither the web nor the app has a reschedule screen for the customer. In CU27 the role is chosen when the account is created; the users list does not change it.

1.2 Actor and Use Case Matrix

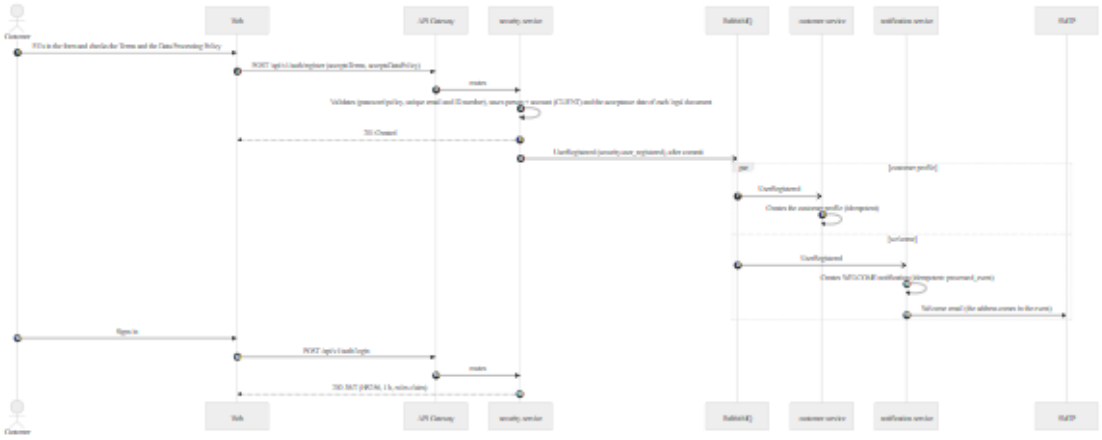
Actor	Use cases
All roles	CU01, CU03, CU04, CU16, CU17, CU18, CU19, CU31
Customer	CU02, CU05, CU06, CU07, CU12, CU14, CU15, CU25, CU26, CU30 (view only)
Operator	CU10, CU11, CU22 (view only)
Administrator	CU09, CU13, CU20, CU21, CU22, CU23, CU24, CU27, CU28, CU29, CU30
System	CU06 (included in CU05), CU08, CU32

The specification of each of the 32 use cases (precondition, main flow, alternative flows, postcondition, status) is in the *Software Requirements Specification*, Tables 23 to 54.

2. Sequence Diagrams

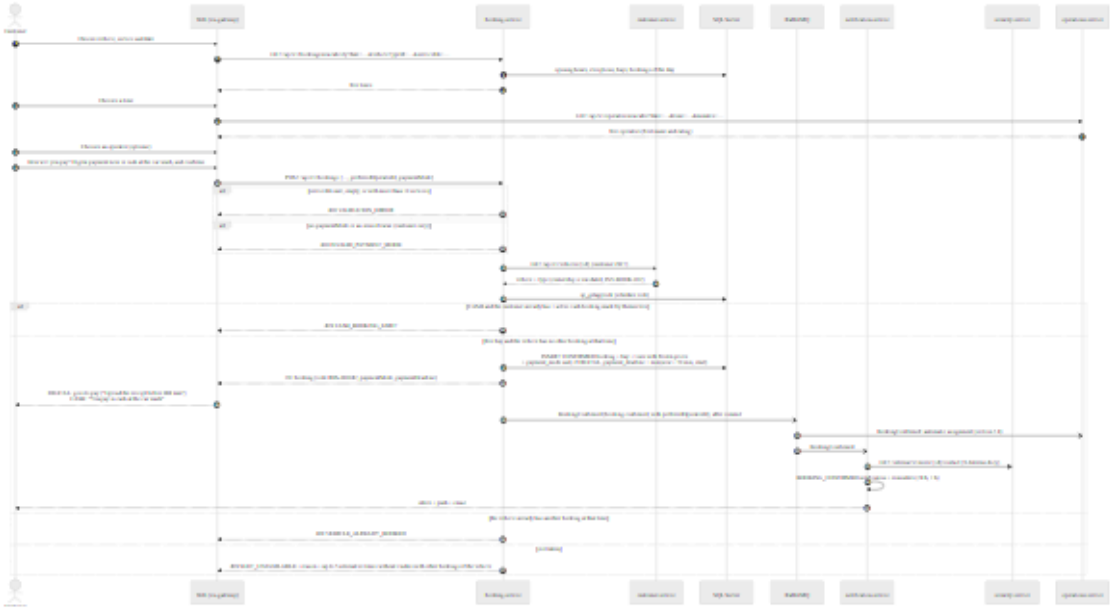
Each diagram follows the path of the request (screen, gateway, services, database and RabbitMQ) with the class and endpoint names from the code. The first five are based on the diagrams in `vehicle-w-docs/08-uml/diagrams/source/`; the next ones cover loyalty, roles and permissions, the account lockout and the inspection report.

2.1 Registration and Welcome (RF-001, RF-018)



Source: `security-service` (`AccountRegistrationService`, `AuthenticationService`, migration `039-security-app-user-legal-consent.sql`), event `security.user_registered`. The web and the mobile app do not send the form until both boxes are checked; the server stores the dates only when both arrive accepted.

2.2 Booking a Wash (RF-005, RF-006, RF-010)



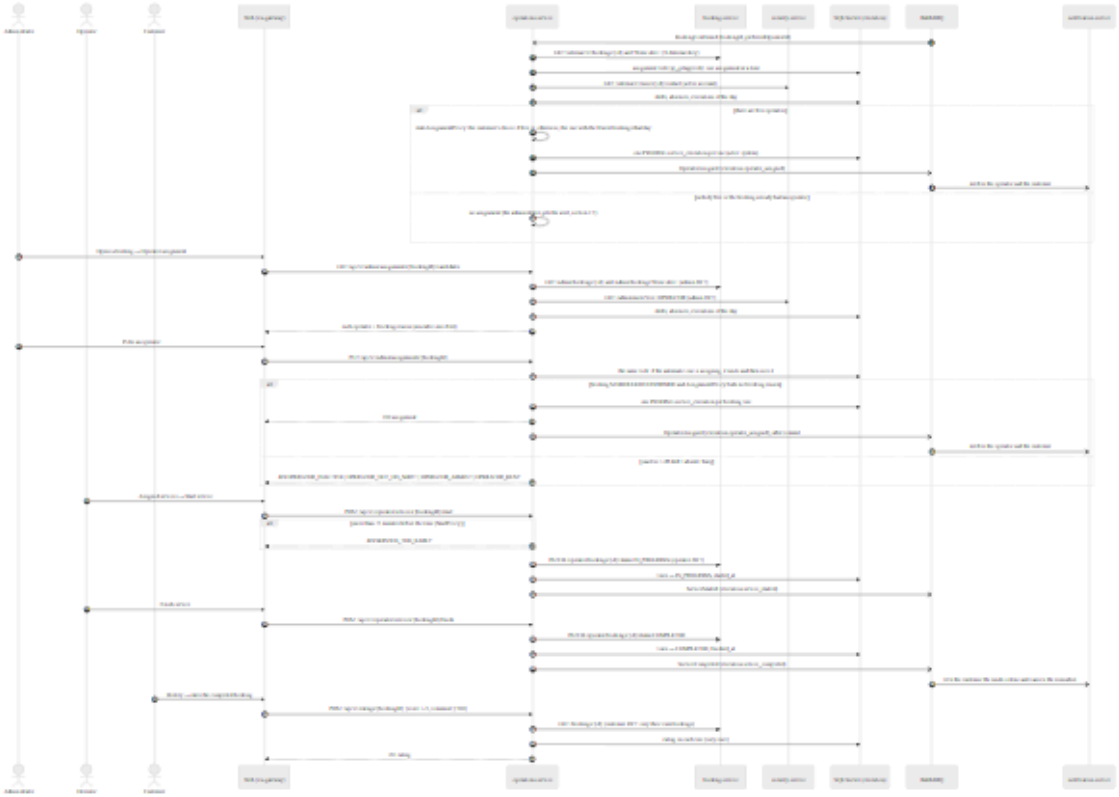
Source: `booking-service` (`BookingService.create`, `Booking.confirmNew`, `AvailabilityPolicy`, `PaymentPolicy`), `operations-service` (`AvailableOperatorsController`), contracts in `07-api/contracts/openapi/`. The cash limit (`BOOKING_CASH_ACTIVE_LIMIT`, 1) is counted with the schedule locked and only with the active bookings (Scheduled, Confirmed, or In progress) made by the customer themselves (`booked_by`). The administrator does not have to send `paymentMode`, and their bookings have no deadline or limit. Since October 27, 2026, a booking has at most 10 services (`BookingDtos.MAX_SERVICES`, also when rescheduling): a null, empty, or longer list returns 400 `VALIDATION_ERROR` instead of 500, and the alternative times leave out those that clash with another booking of the same vehicle (`AvailabilityPolicy.withoutBusyTimes`).

2.3 Reminders and Alert for Bookings Without an Operator (ADR-011)



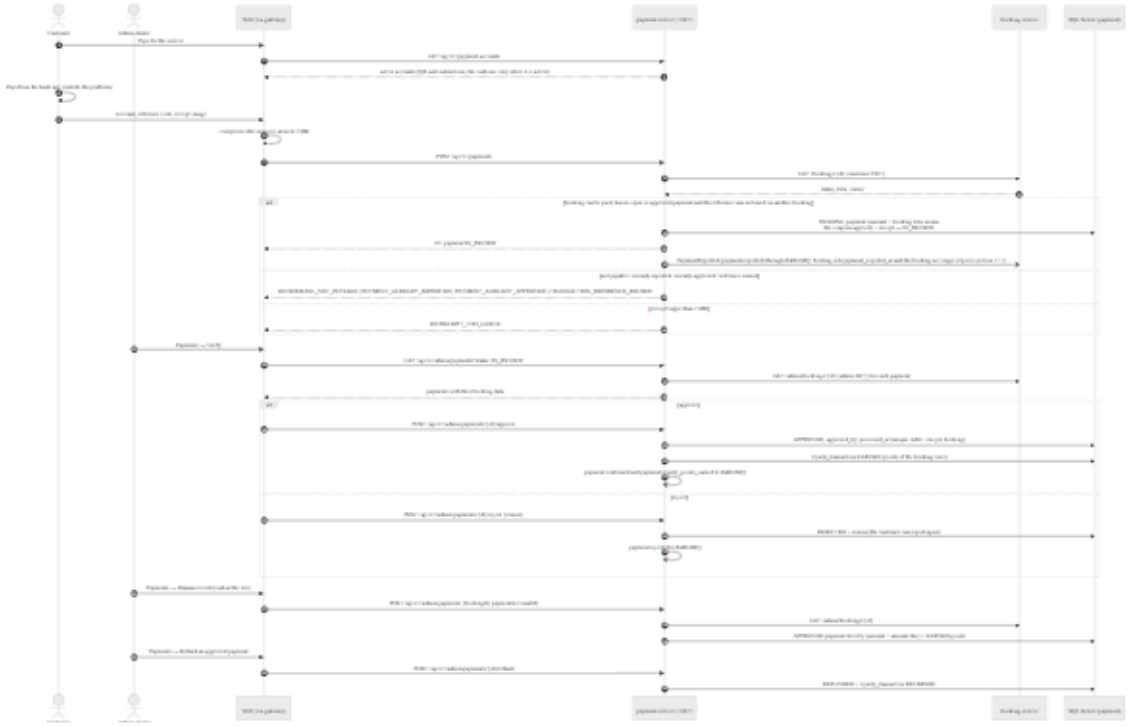
Source: `notification-service` (`BookingReminderService`, `ReminderScheduler`, `StaffNotificationService`, migration 031).

2.4 Operator Assignment, Execution and Rating (RF-008, RF-009, RF-010, RF-011, RF-012, RF-012b)



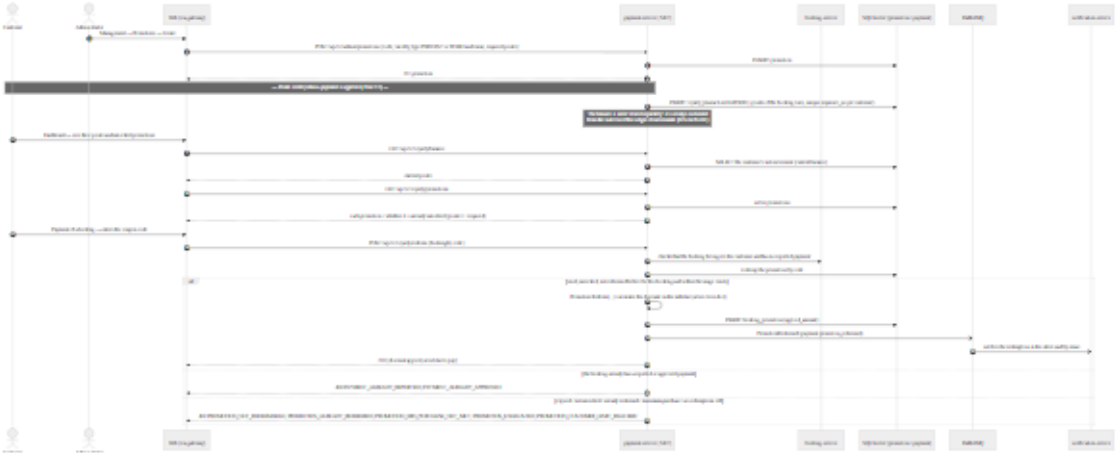
Source: `operations-service` (`BookingConfirmedListener`, `OperationsService.autoAssign / candidates / assign / start / finish / rate`, `ExecutionPersistenceAdapter.lockAssignments`, `AssignmentPolicy`, `AutoAssignmentPolicy`, `StartPolicy`, ADR-014). The lock prevents a simultaneous manual and automatic assignment from giving the same operator two services at the same time.

2.5 Payment by Receipt and Review (RF-013, RF-013b, RF-014)



Source: `payment-service` (`PaymentCommandService.ReportAsync / ApproveAsync / RejectAsync / RegisterInPersonAsync / RefundAsync`, `BookingPaymentGuard`, `LoyaltyRewardService`, `Payment` aggregate, ADR-008, ADR-013). notification-service turns each `payment.*` event into a notice for the customer.

2.6 Coupon Redemption and Loyalty Points (RF-024, RF-025, RF-026)



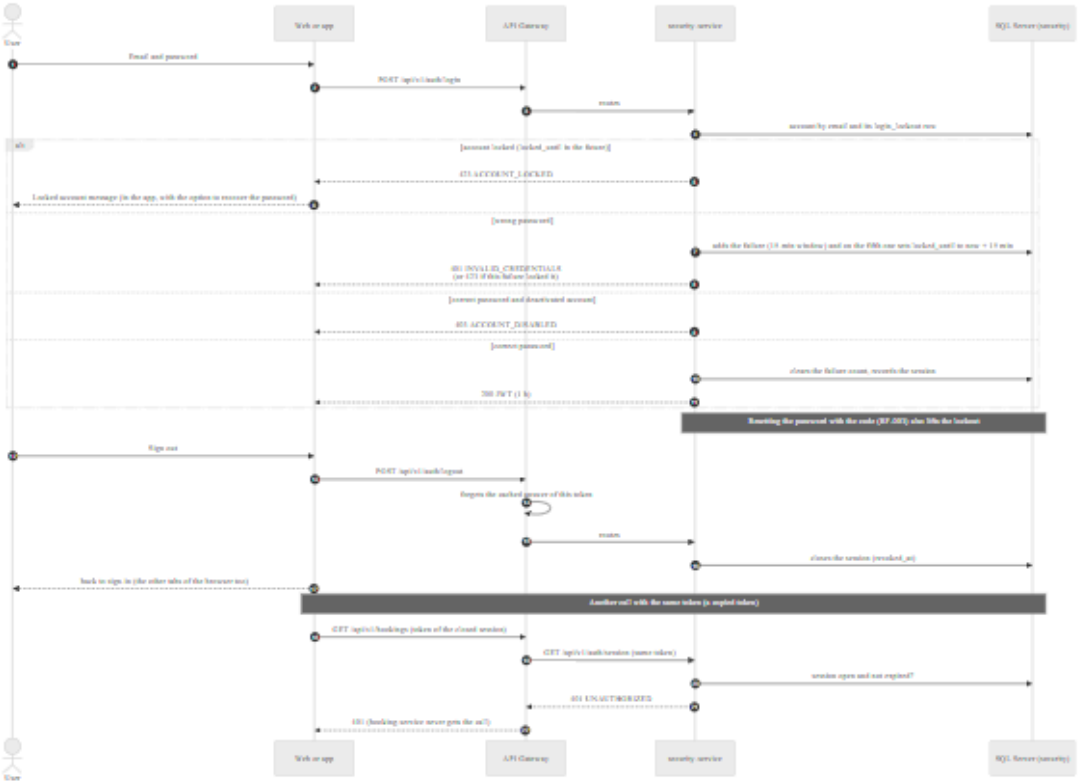
Source: `payment-service` (`LoyaltyController`, `CustomerPromotionService.RedeemAsync`, `Promotion.Redeem` aggregate, `LoyaltyLedgerRepository`), ADR-015.

2.7 Roles and Permissions Management (RF-029)



Source: `security-service` (`RolePermissionController`, `RolePermissionAdministrationService`, migration `026-security-role-permissions.sql`), ADR-015.

2.8 Sign-in With Lockout After Failed Attempts and Sign-out (RF-002, RF-004b, RF-027)



Source: `security-service` (`AuthenticationService.login / logout / isActive`, `ActiveSessionValidator`, `LoginLockout`, `PasswordRecoveryService`, migration `034-security-login-lockout.sql`) and the gateway (`SessionCheckFilter`). The gateway keeps each answer of `/api/v1/auth/session` (if security-service does not answer within 3 seconds, it lets the request through: *fail-open*) for 30 seconds per token, so an open session costs one extra call every half minute, not one per request. A session also closes when the administrator deactivates the account or the owner closes it; that reaches the other services within the same 30 seconds.

2.9 Vehicle Inspection Report (RF-036)



Source: `operations-service` (`InspectionReport`, migration `033-execution-inspection-report.sql`), `notification-service` (shared report notice), web (`features/admin/pages/inspection`), and mobile app (`AdminInspectionScreen`, `InspectionService`). In the app the photos are taken with the camera or picked from the gallery and shrunk on the phone before the `POST /findings`; the **Publish and share link** button opens the phone share menu.

2.10 Password Recovery With a 6-digit Code (RF-003)



Source: `security-service` (`PasswordRecoveryService`, `PasswordResetToken`, `PasswordRecoveryPolicy` with `code-ttl: 15m` and `max-failed-attempts: 5`, `SecureRandomResetCodeGenerator`, `EmailResetCodeNotifier`). The *forgot* answer does not reveal whether the email exists.

2.11 Expiry of the Digital Payment (RF-005, RF-033, RN-BOOK-03)

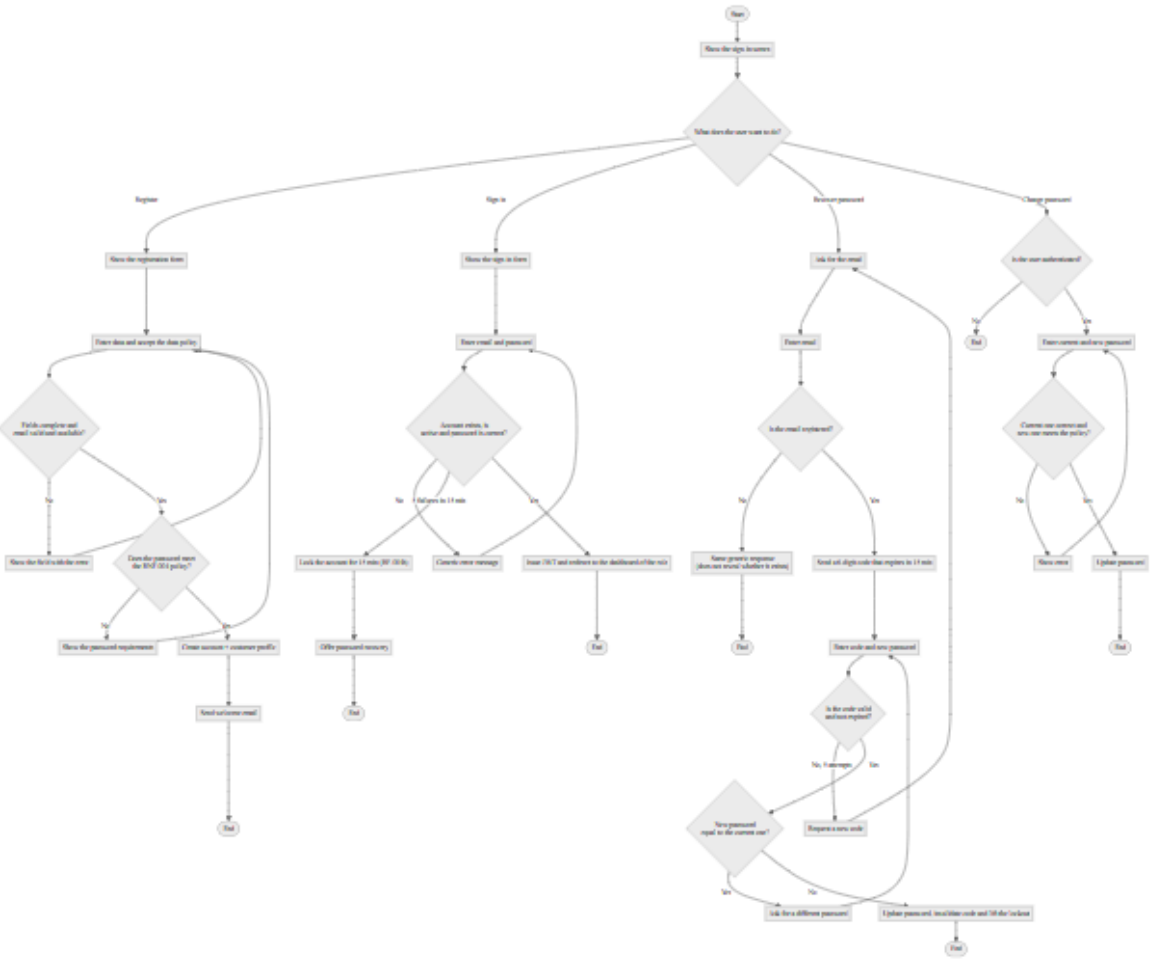


Source: `booking-service` (`PaymentEventsListener`, `RabbitConsumerConfig`, `PaymentDeadlineJob`, `BookingService.cancelForPaymentTimeout`, `PaymentServiceDirectory`, `PaymentTimeoutOutcome`) and `payment-service` (`InternalPaymentsController`, `InternalApiKeyAuthenticationHandler`). Since there is no Outbox, a `payment.reported` event can be lost; that is why the job asks payment-service before cancelling (AC-89). For payment, `reported` is true if the booking has an `IN_REVIEW` or `APPROVED` payment. The job is turned off with `BOOKING_PAYMENT_EXPIRY_ENABLED=false`, and the interval is `BOOKING_PAYMENT_EXPIRY_INTERVAL` (60 s).

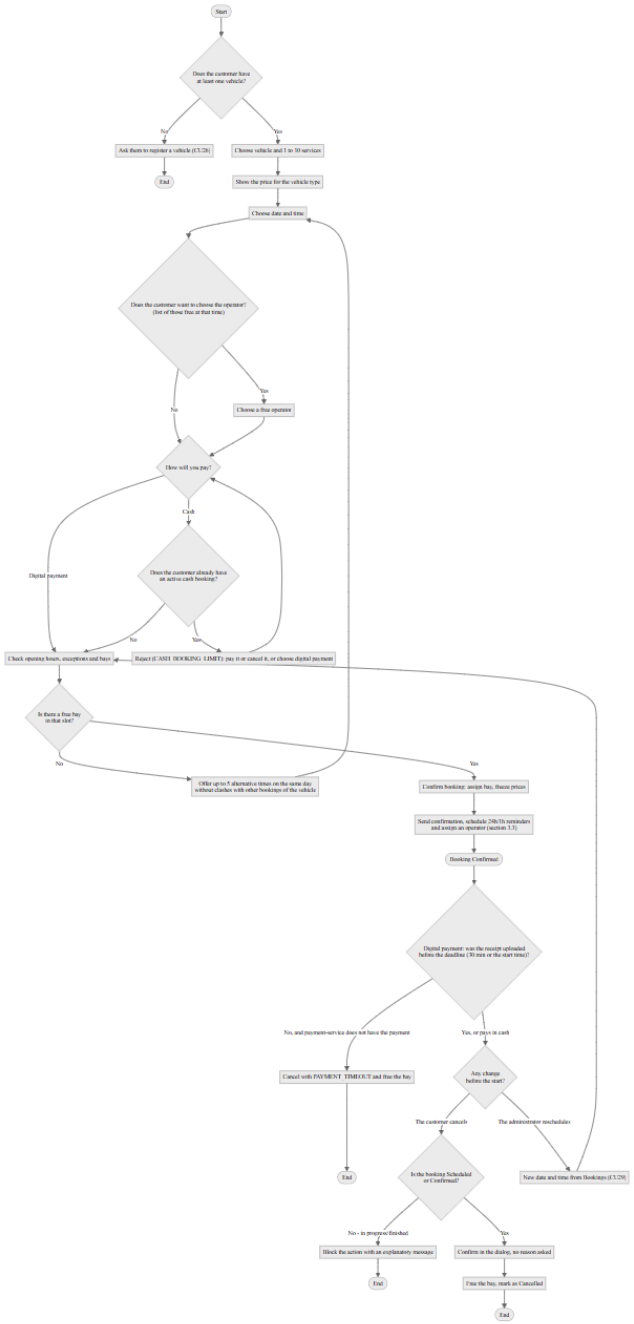
3. Activity Diagrams

The activity diagrams show the business processes as they work in the system. There is no payment gateway: the customer uploads a receipt and the administrator reviews it (ADR-008).

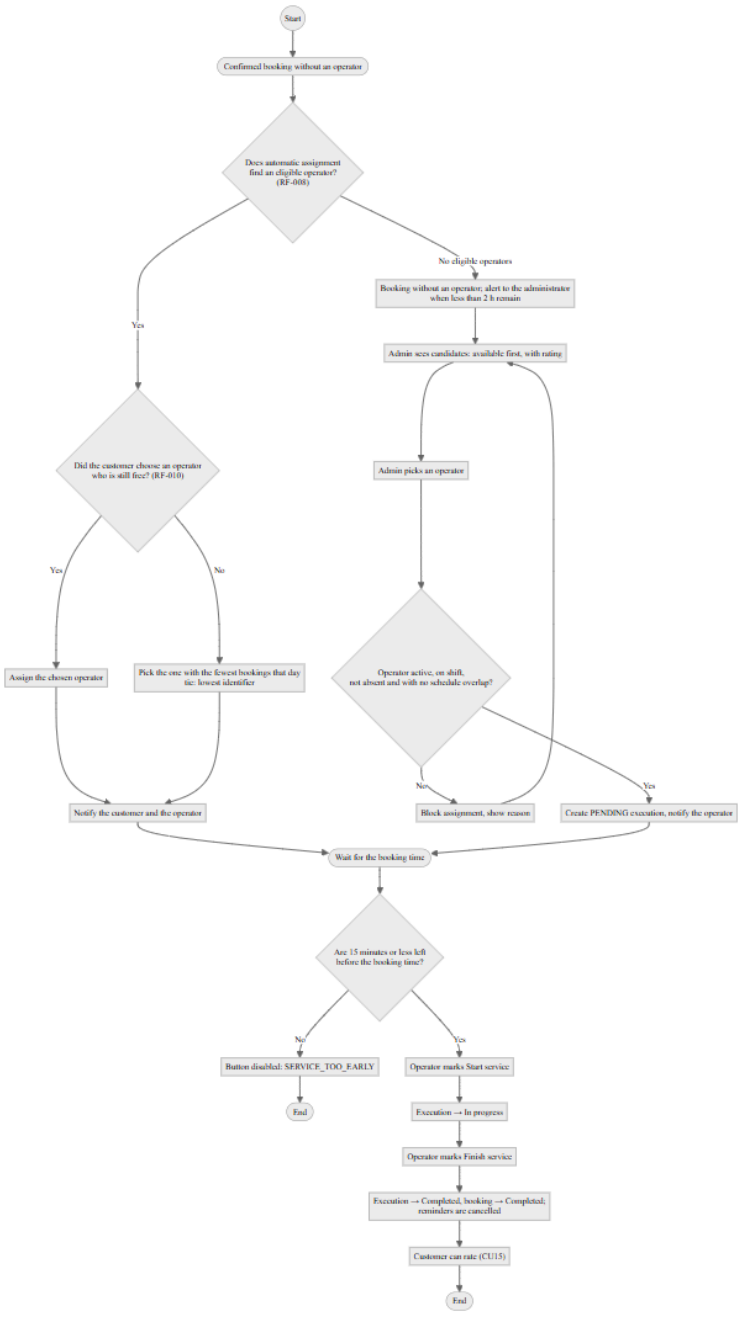
3.1 Authentication (RF-001 to RF-004)



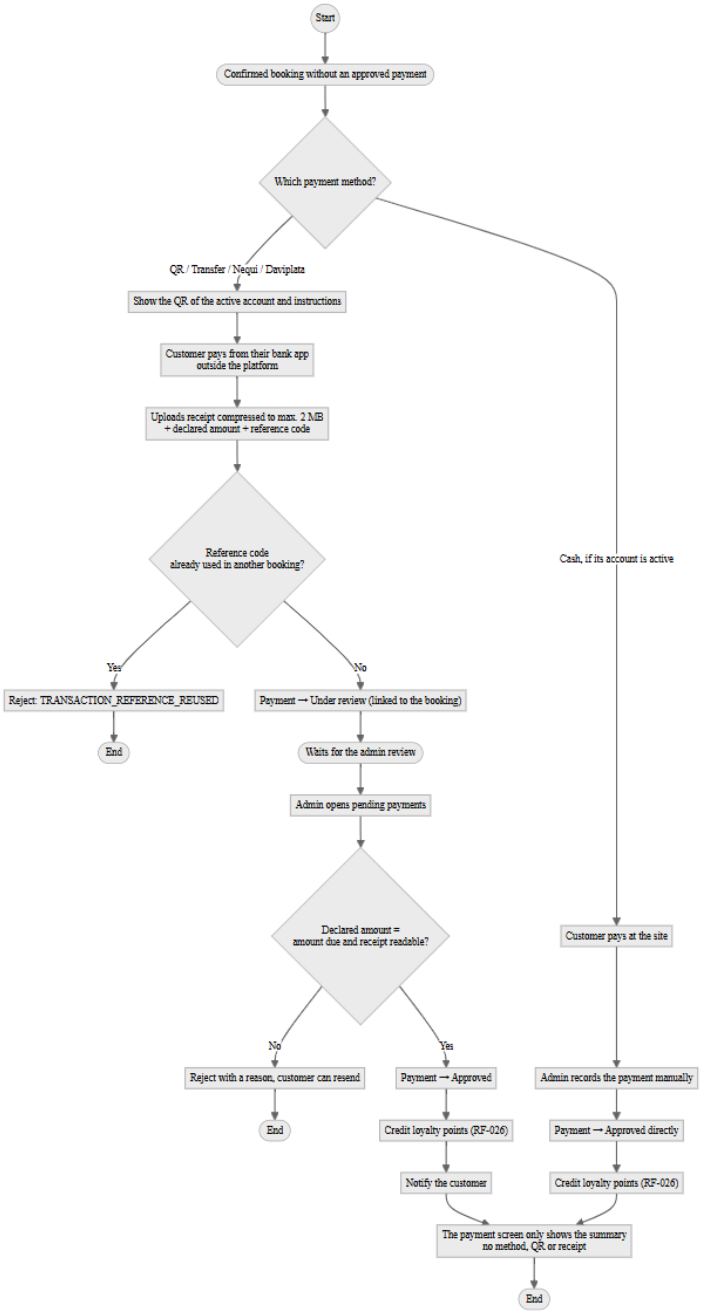
3.2 Booking a Wash (RF-005, RF-006, RF-007, RF-010)



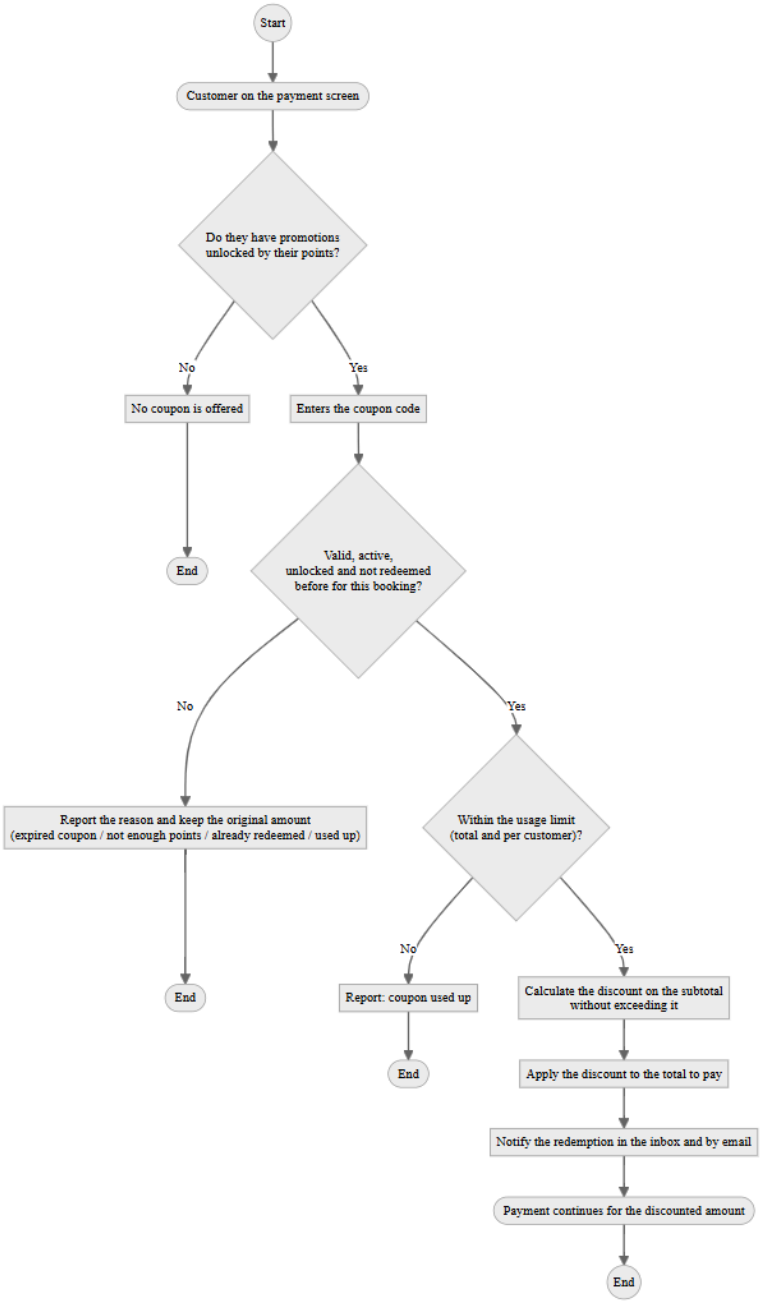
3.3 Operator Assignment and Service Execution (RF-008, RF-009, RF-010, RF-011, RF-012b)



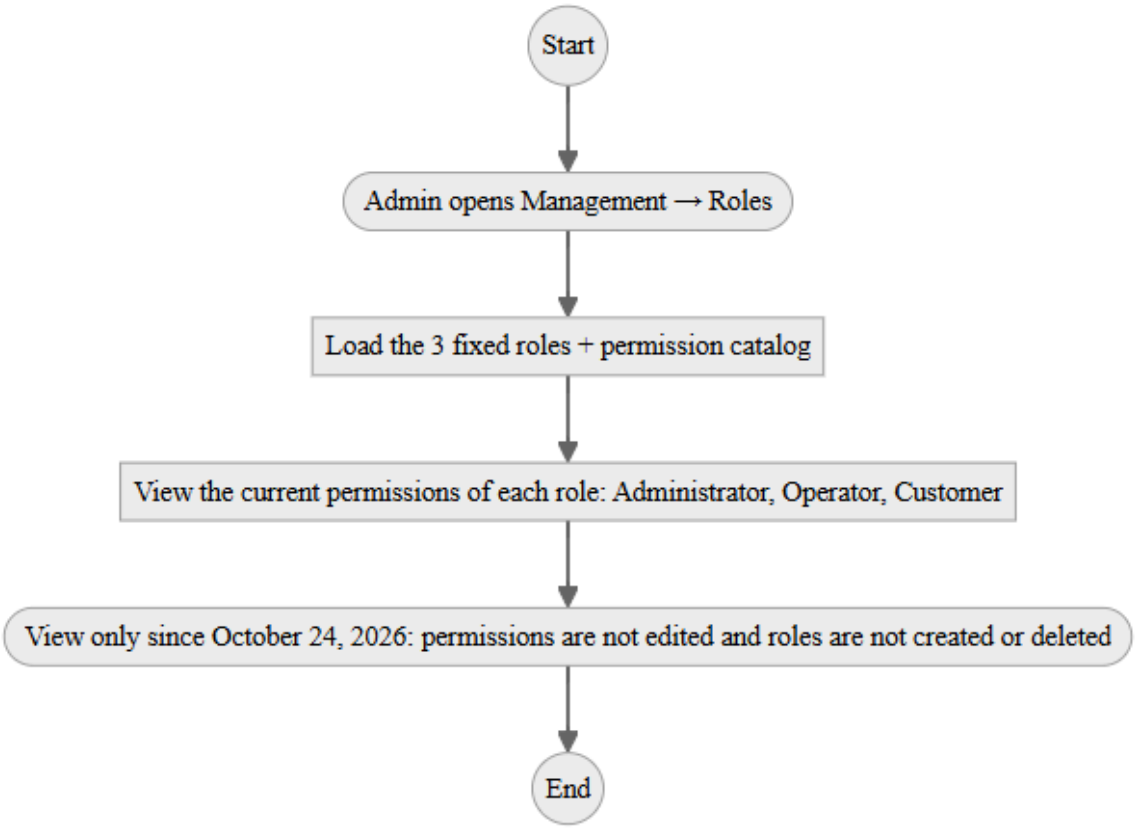
3.4 Paying for a Booking (RF-013, RF-013b, RF-014, RF-016)



3.5 Redeeming a Loyalty Coupon (RF-024, RF-025, RF-026)

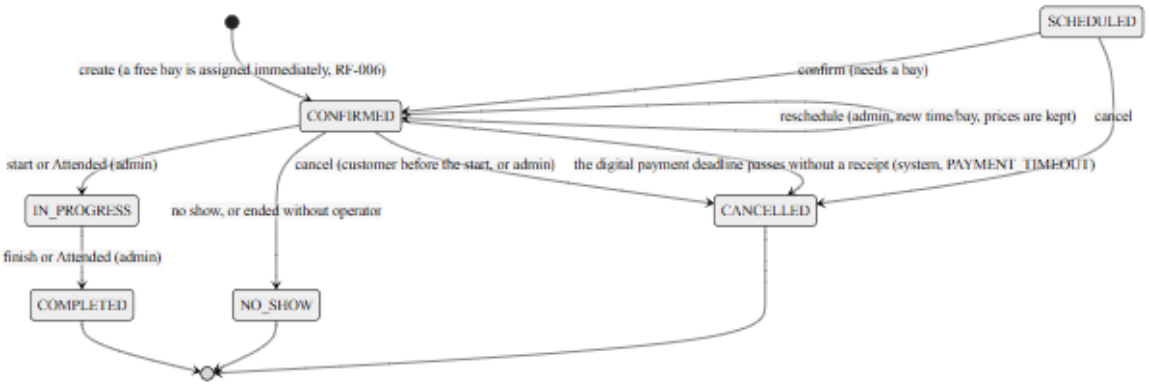


3.6 Managing Roles and Permissions (RF-029)



4. State Diagrams

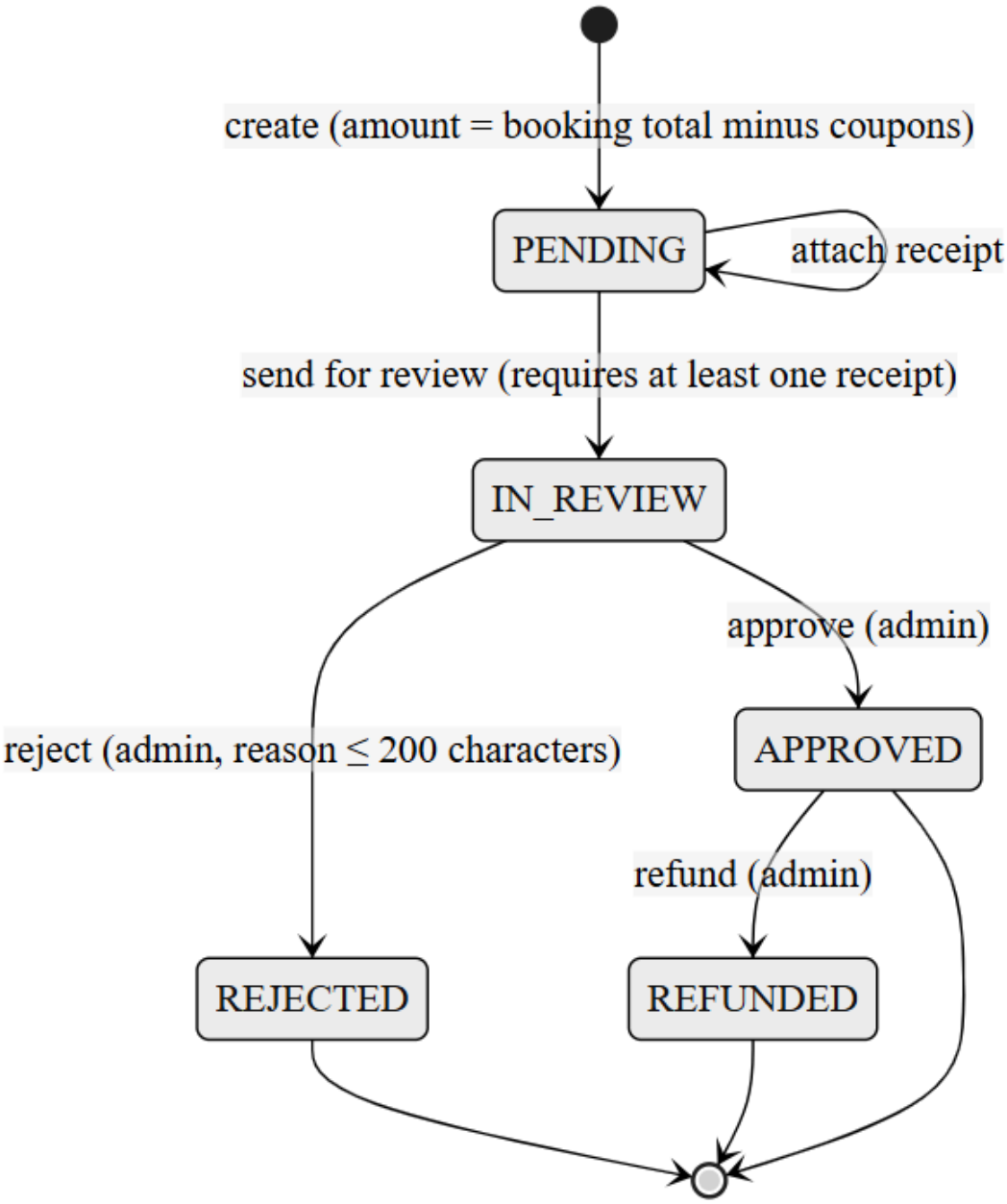
4.1 Booking Life Cycle (RF-006, RF-007)



Source: `booking-service` (`Booking.moveTo`, `Booking.cancel`, `BookingStatus`) and `operations-service` (`FollowUpJob`, `FollowUpPolicy`). *Bookings that ended and nobody closed:* the system only decides on its own when there is no doubt. Until October 23, 2026 `ExpiredBookingsJob` cancelled bookings still `SCHEDULED`; it was removed on October 24 (B-07) because it never acted: every booking is created and rescheduled as `CONFIRMED`. Every 5 minutes, the operations `FollowUpJob` asks booking (`POST /internal/v1/bookings/{id}/no-show`) to set to `NO_SHOW` a `CONFIRMED` booking that never had an operator and ended more than an hour ago; booking checks again that it expired and, if not, answers 409 `BOOKING_NOT_EXPIRED`. A `CONFIRMED` booking with an operator that nobody started (`NOT_STARTED`) and an `IN_PROGRESS` one that nobody finished one hour after its end (`NOT_FINISHED`) **never** change status on their own: they go to the administrator's "To close" list, and the administrator resolves them with "Attended" (the booking goes through `IN_PROGRESS` and ends `COMPLETED`, without publishing `ServiceCompleted`) or with "No-show" (only if it was never started). This avoids recording as a no-show a service that was actually done and that the operator forgot to mark. There is never an automatic refund: payments are reviewed by the administrator. Only `SCHEDULED` and `CONFIRMED` can be rescheduled or cancelled (RF-007). The customer only cancels, before the start time and

without writing a reason (the booking stores `CUSTOMER_REQUEST`); the administrator reschedules (CU29) and may choose the cancellation reason. `SCHEDULED` exists in the model, but the current flow always creates the booking directly as `CONFIRMED`. Since October 26, 2026, `PaymentDeadlineJob` cancels with the reason `PAYMENT_TIMEOUT` a booking with digital payment that reached its deadline without a receipt, only if payment-service confirms it does not have the payment (section 2.11); rescheduling changes neither the payment mode nor the deadline.

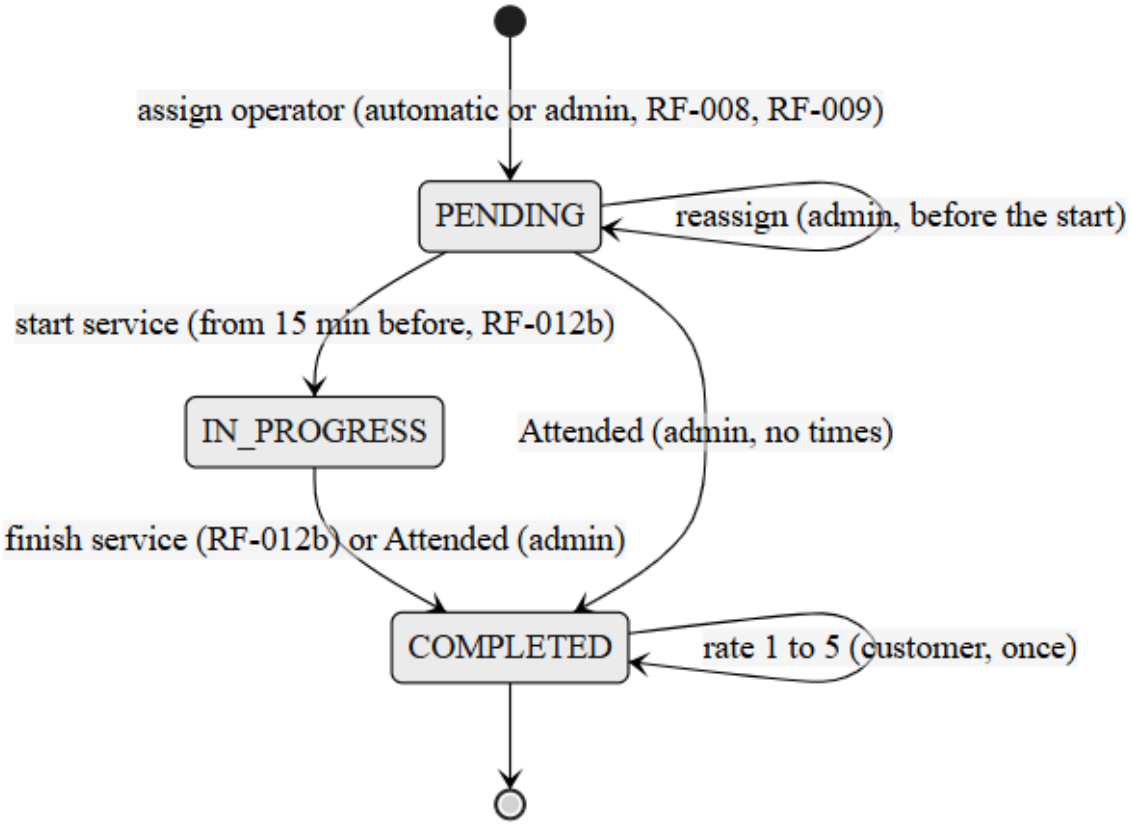
4.2 Payment Life Cycle (RF-013b, RF-014, ADR-008)



Source: `payment-service`, `Payment` aggregate. A booking allows only one approved payment and only one open payment at a time; after `REJECTED` the customer can report a new

payment. A customer payment goes from `PENDING` to `IN_REVIEW` in the same request; the cash payment the administrator registers is `APPROVED` at once.

4.3 Service Execution Life Cycle (RF-012b)

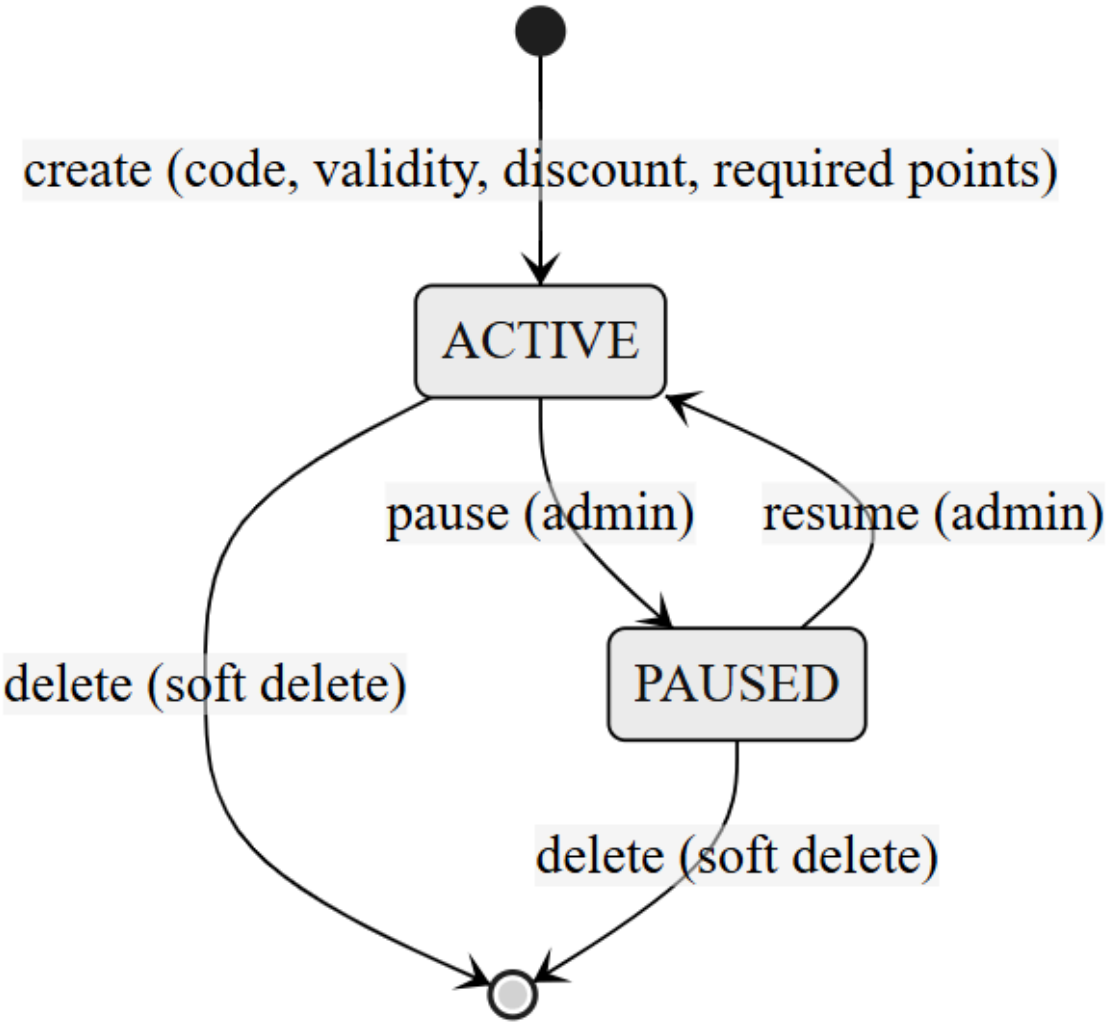


Source: `operations-service` (`ServiceExecution`, `StartPolicy`, `FollowUpPolicy`).

`COMPLETED` enables the customer's rating (RF-012). When the administrator closes a service from "To close" as "Attended", the lines end `COMPLETED` without made-up times: with no start time they do not count for punctuality and with no end time they add no worked hours, but they do count as a service done and as revenue. The operator can still start the same service late, because starting has no upper limit, and then the real times count. If a service with an operator is not started or not finished on time, operations notifies the operator and every active administrator

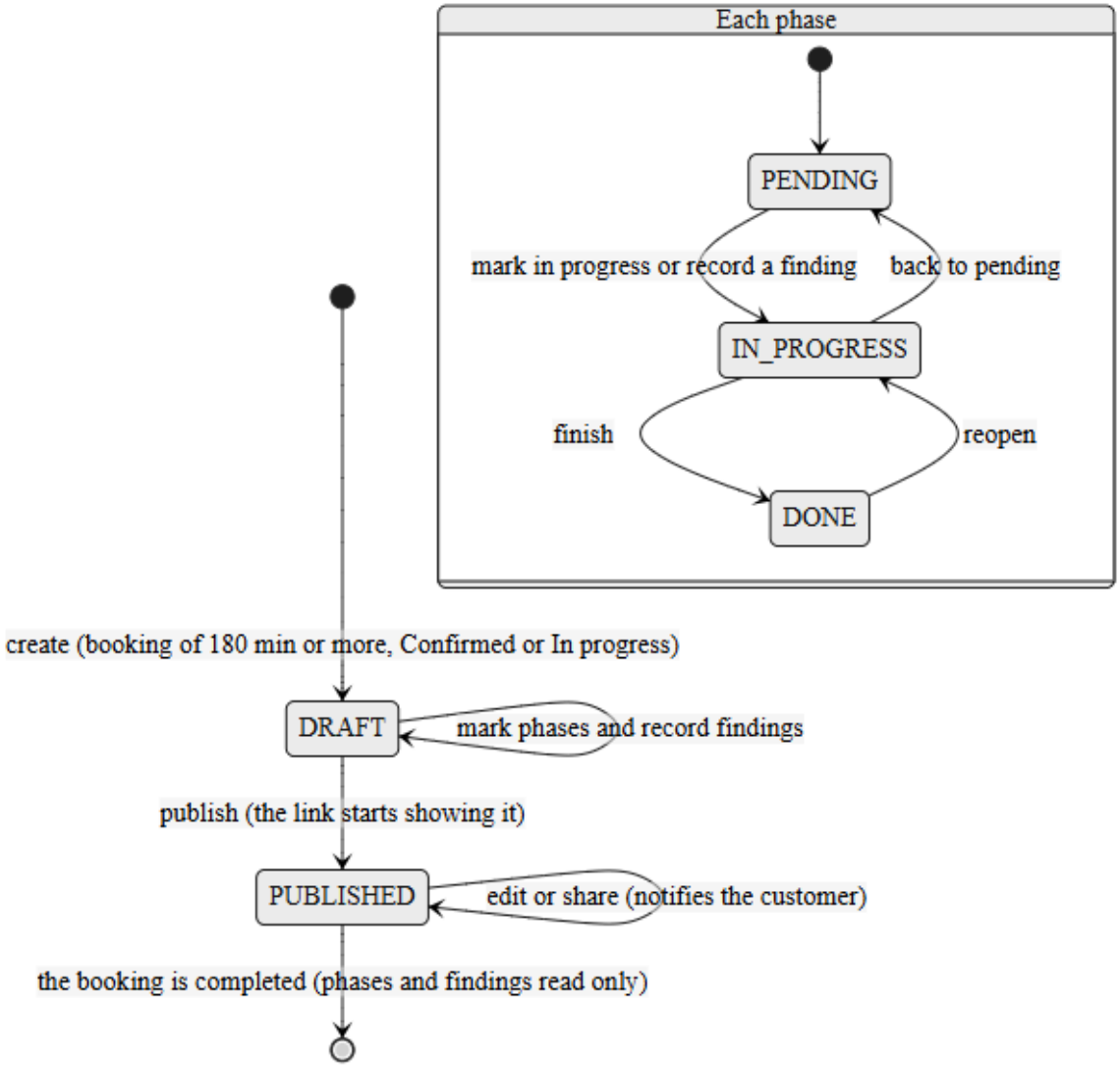
once (inbox and push, migration 040). `PAUSED` and `WITH_ISSUE` exist in the `execution.execution_status` catalog, but no use case uses them today.

4.4 Promotion Life Cycle (RF-024)



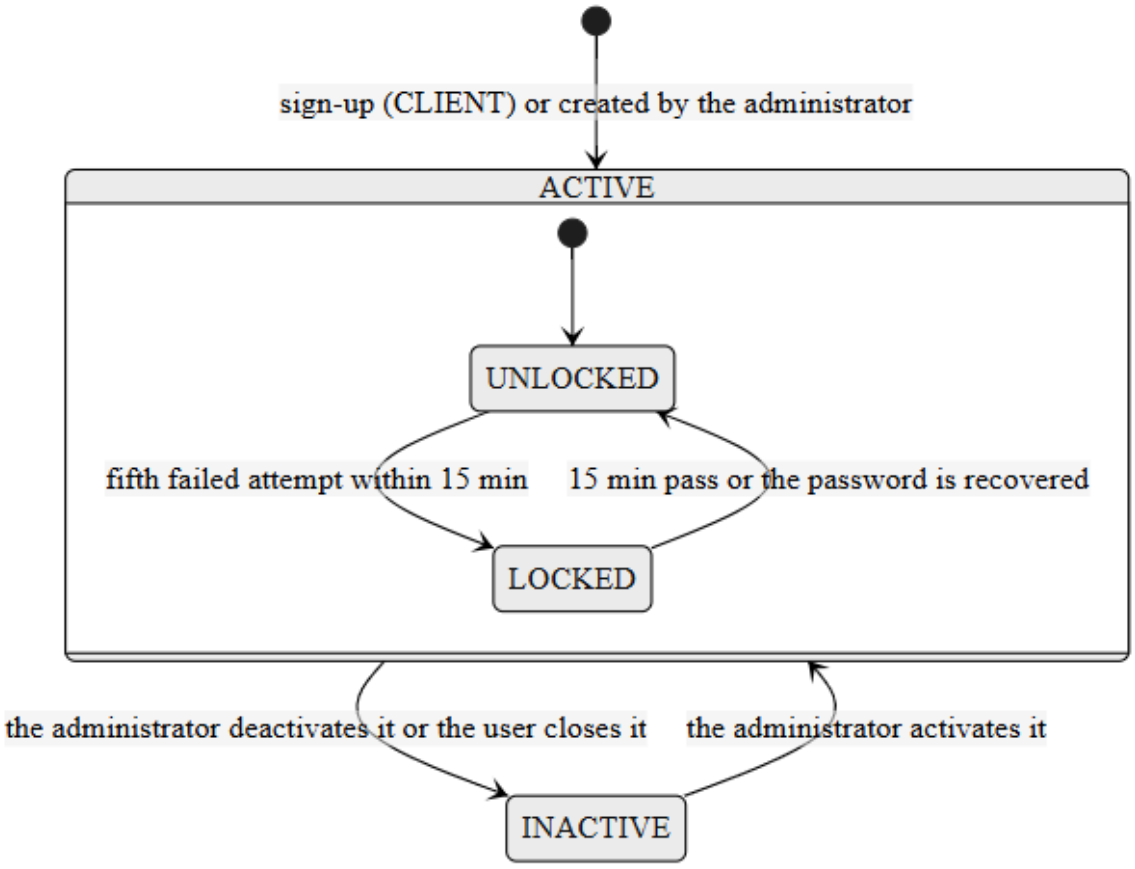
Source: `payment-service`, `Promotion.SetActive`, `Promotion.StatusFor`, `Promotion.IsAvailableOn`. The only thing stored is whether it is active or paused; the administrator's screen shows "scheduled" until the start date arrives. Only an active promotion within its validity period can be redeemed (RF-024, RN-PROMO-01): after the end date it is no longer offered or redeemed, although no "expired" status is stored.

4.5 Inspection Report and Its Phases (RF-036)



Source: `operations-service`, `InspectionReport` (`publish`, `requirePublished`, `setPhaseStatus`, `addFinding`). The phases are reception, exterior, interior, engine and delivery; only a published report can be shared, and it is edited while the booking is still Confirmed or In progress.

4.6 User Account (RF-002, RF-004b, RF-027, RF-028)

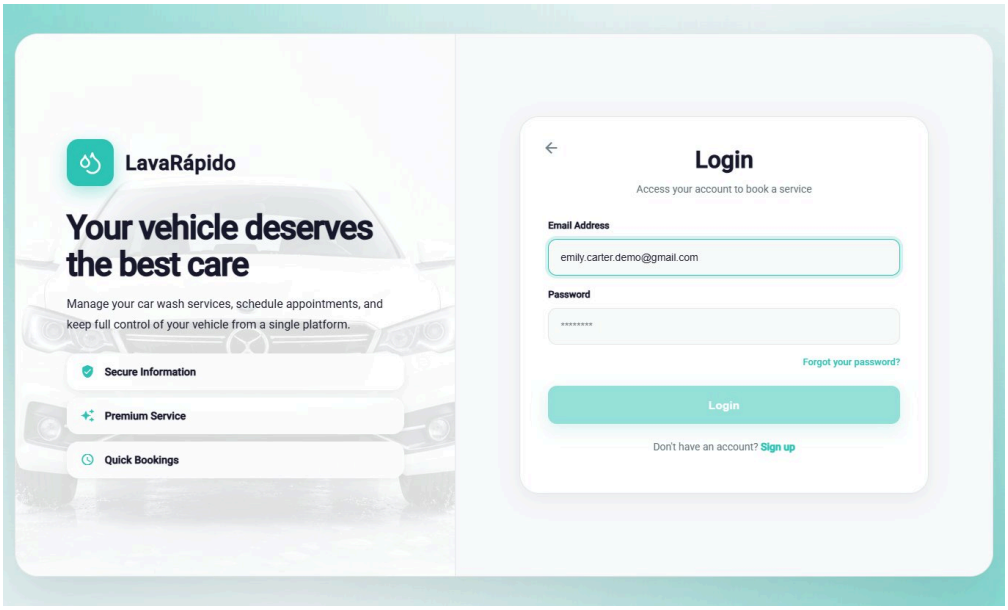
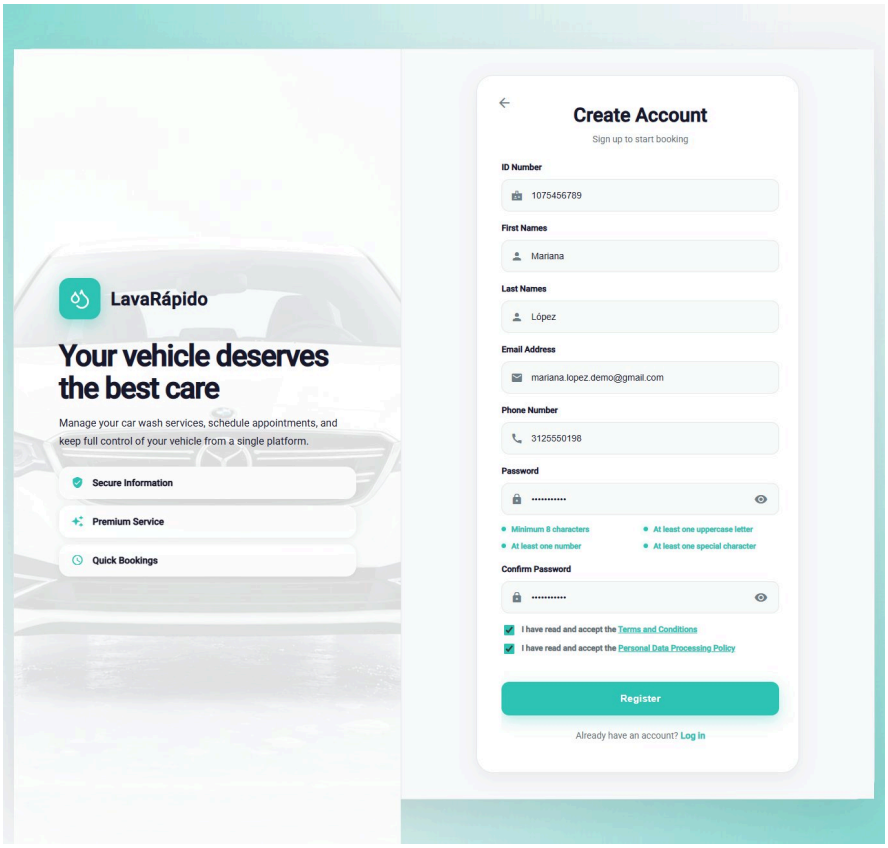


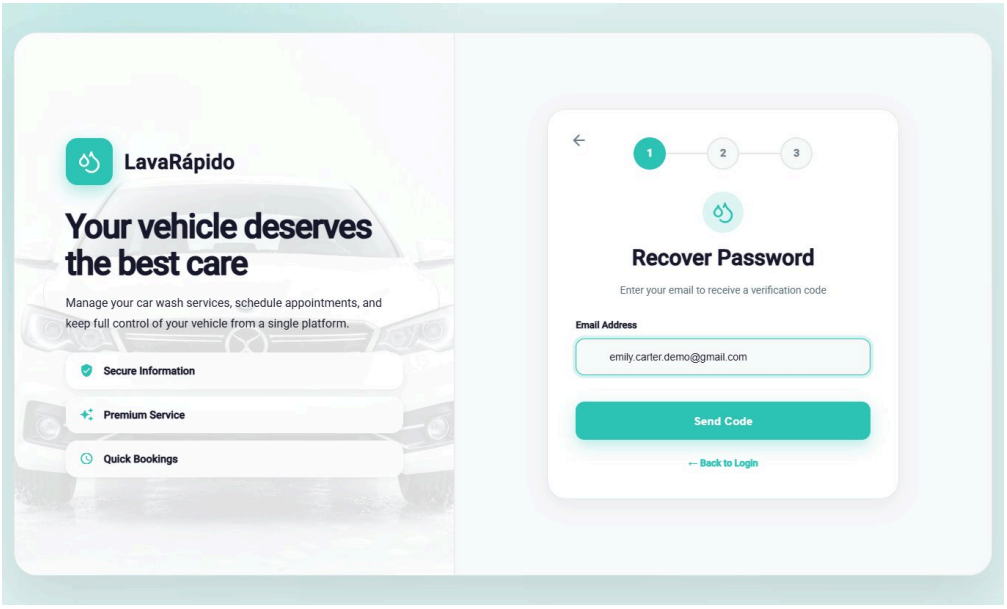
Source: `security-service` (`UserAccount`, `LoginLockout`, `UserAdministrationService.changeStatus`, `AccountDeactivationService`). A successful sign-in before the fifth attempt clears the failure count. The user closes their account with the password. A locked account answers 423 even with the right password; an inactive one answers 403 after the password is checked. No administrator account can be deactivated (403 `ADMIN_CANNOT_BE_DISABLED`), and an administrator cannot deactivate or close their own account (403 `CANNOT_DISABLE_OWN_ACCOUNT`). Each account has one role, set when it is created: there is no role change. When it becomes inactive, all its sessions are closed and their tokens stop working within 30 seconds (section 2.8).

5. Screen Inventory (Mockup)

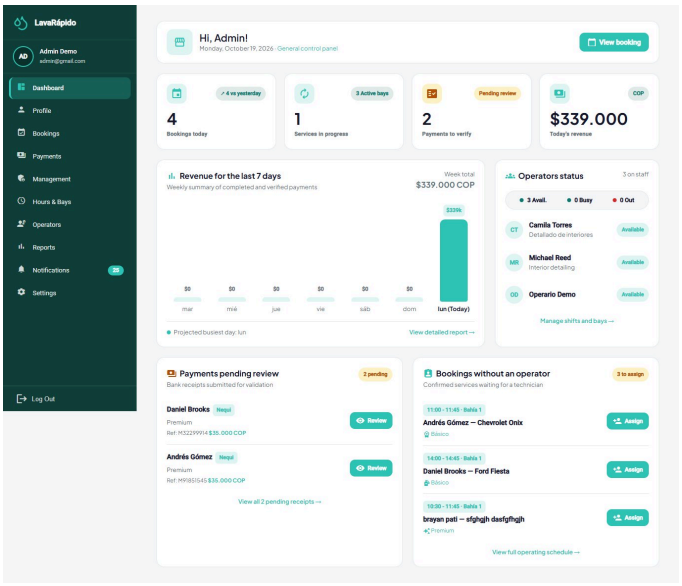
Screen by screen and by role, including the create and edit modals of each module. All screenshots are in the `capturas/` folder of this document. The Management → Users screenshot was taken again on October 20, 2026: the table only shows the user type and has the Enabled and Disabled filters. Since October 20, clicking a notification or a booking of the history opens its detail, with a button to go to the related section (for example, **Go to payment**).

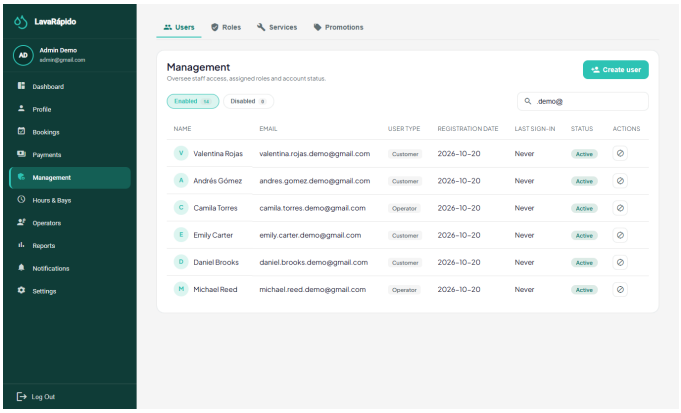
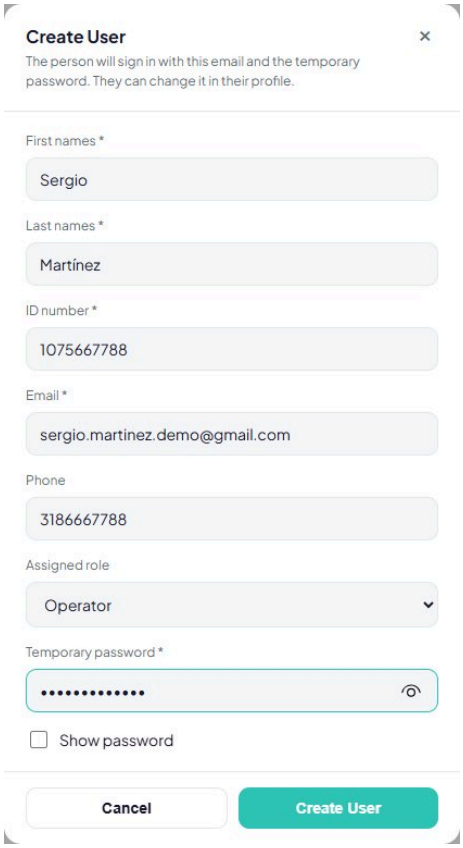
5.1 Common Screens (No Session)

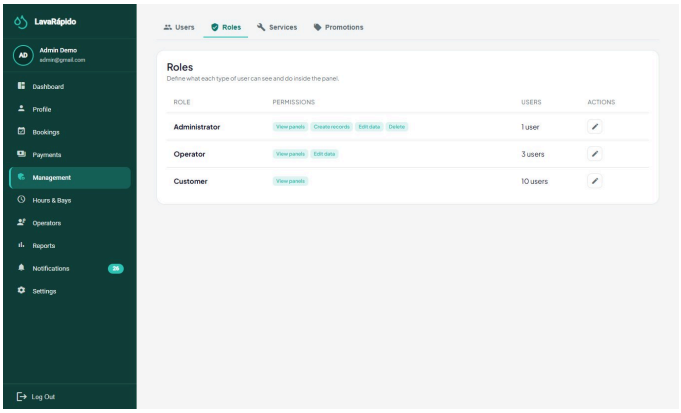
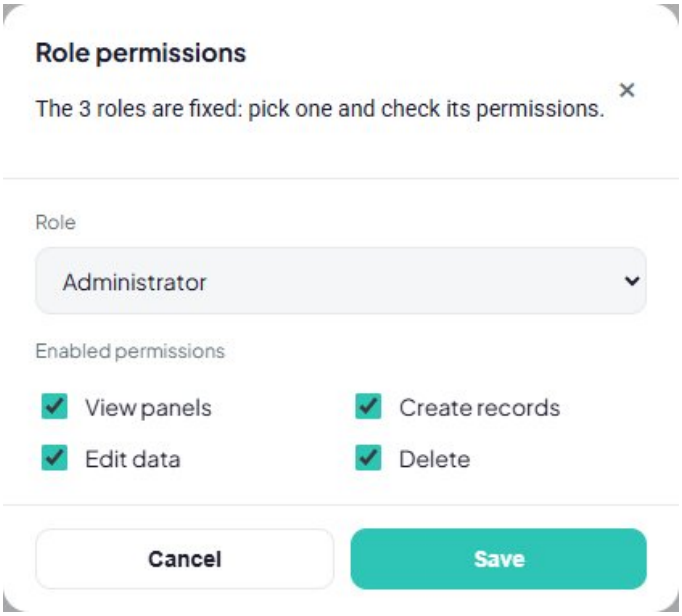
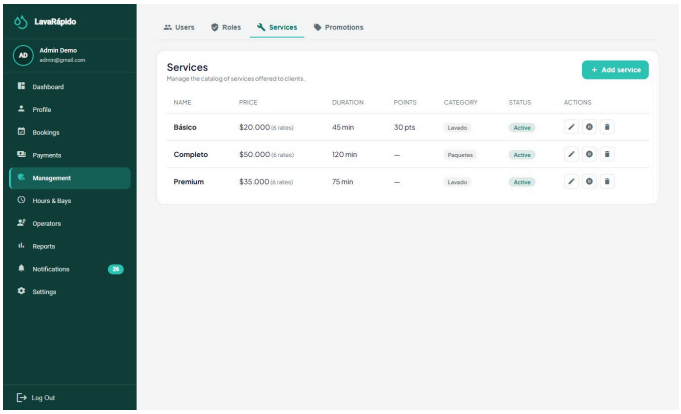
Screen	Evidence
Sign-in	
Registration	

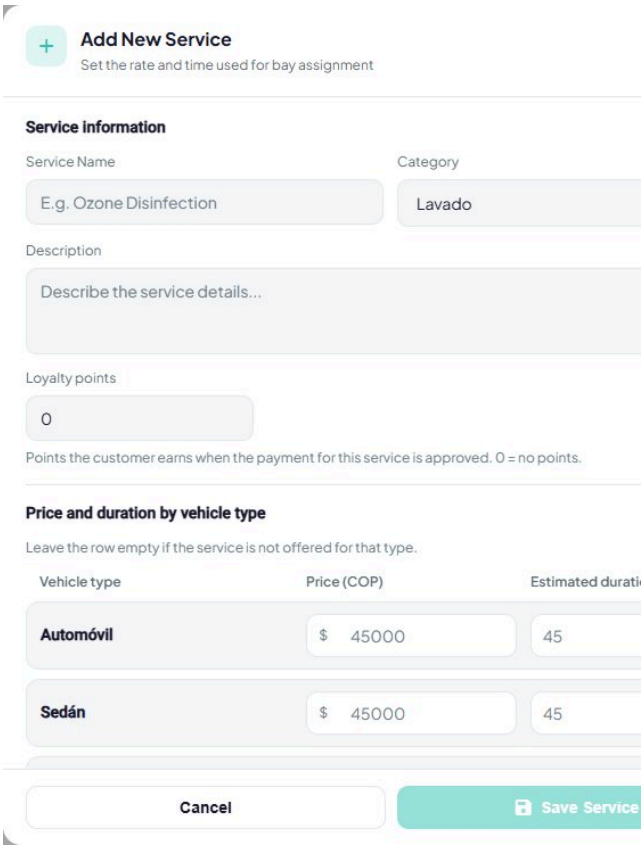
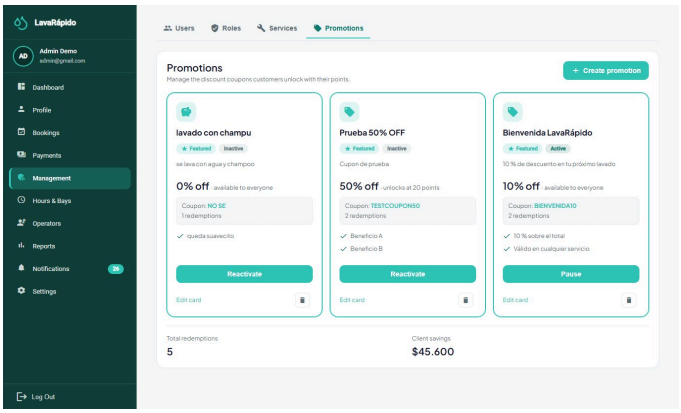
Screen	Evidence
Password recovery	

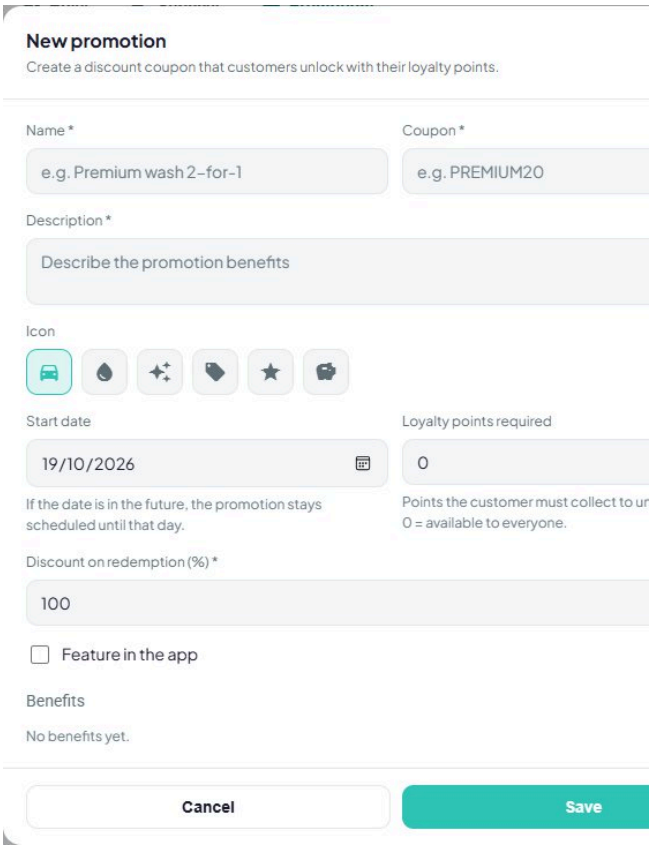
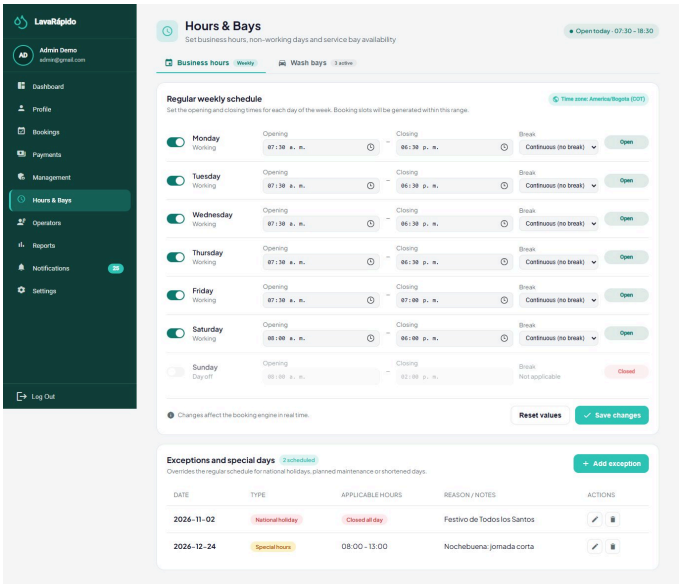
5.2 Administrator Role

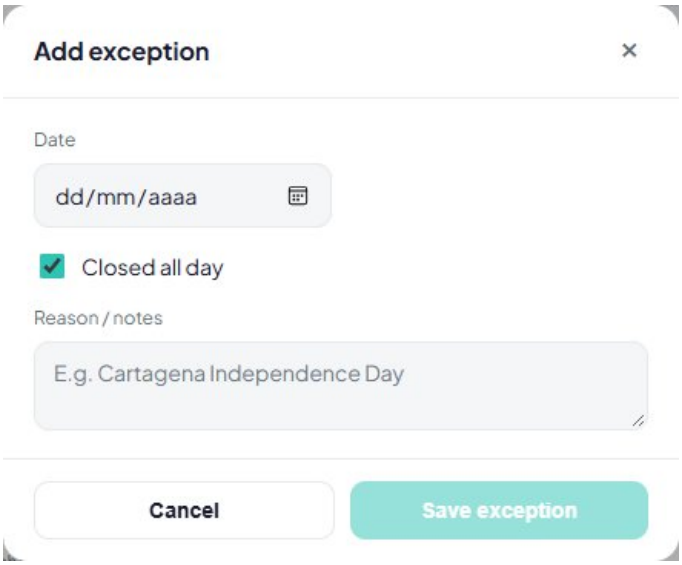
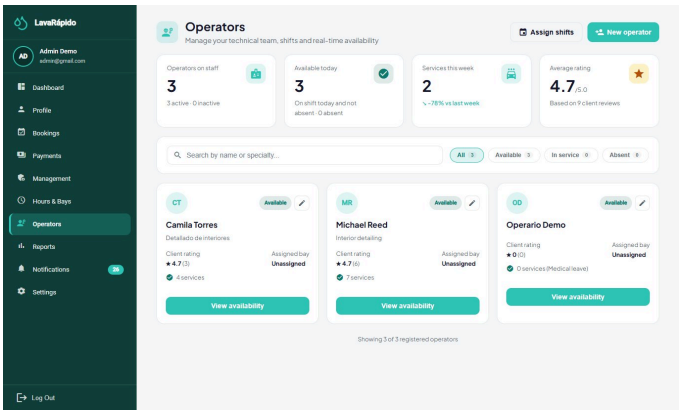
Screen	Evidence
Dashboard	

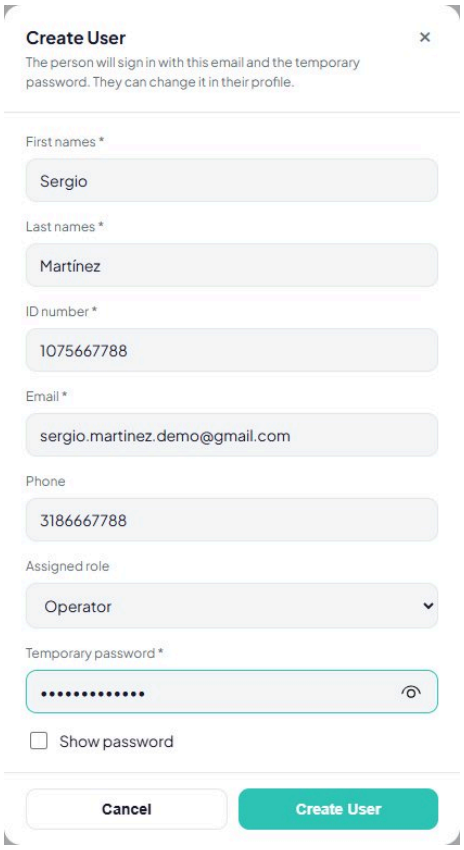
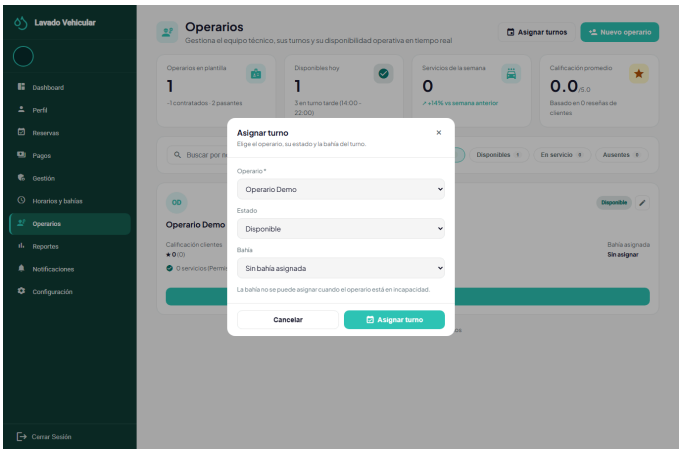
Screen	Evidence
Management → Users	
Modal: Create user	

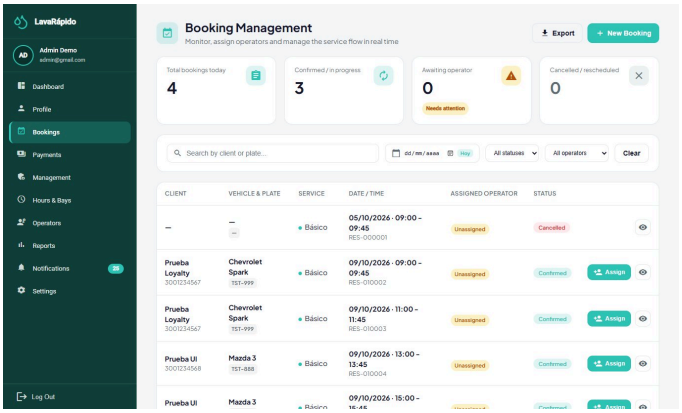
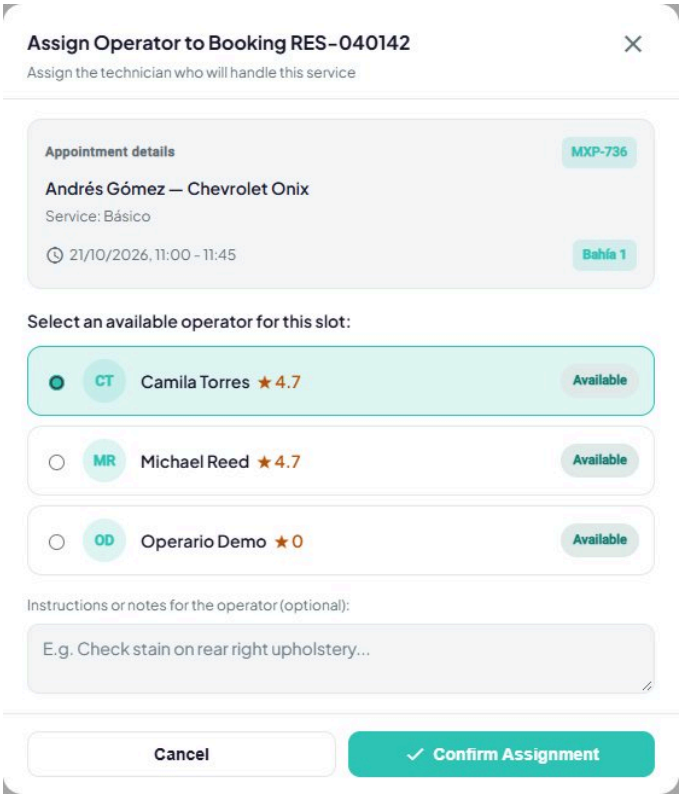
Screen	Evidence
Management → Roles	
Modal: Edit role permissions (removed on October 24, 2026: the matrix is read-only)	
Management → Services (catalog)	

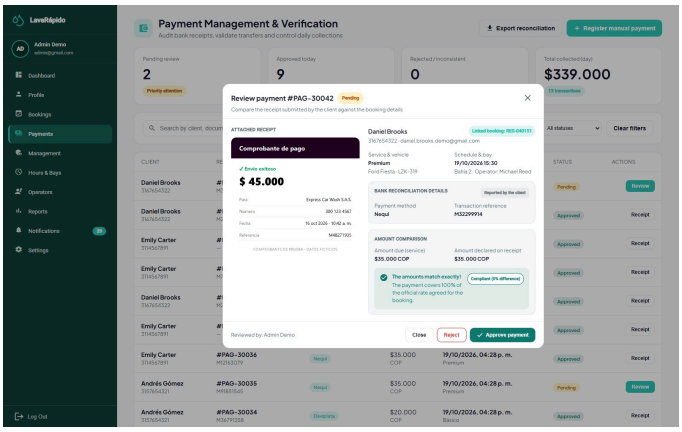
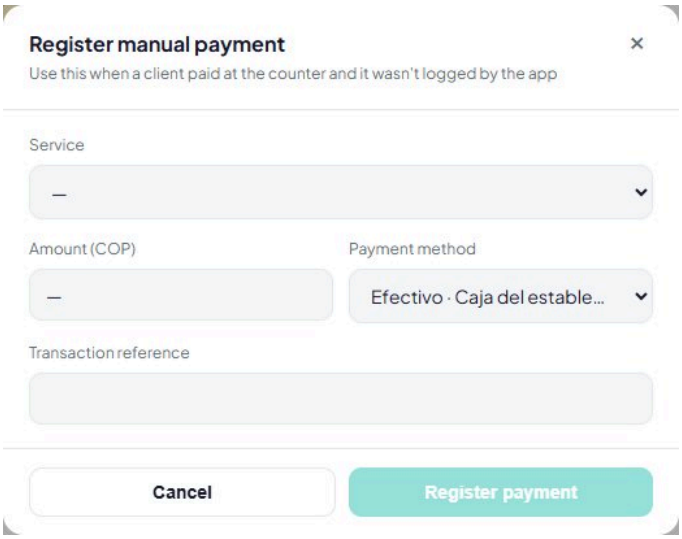
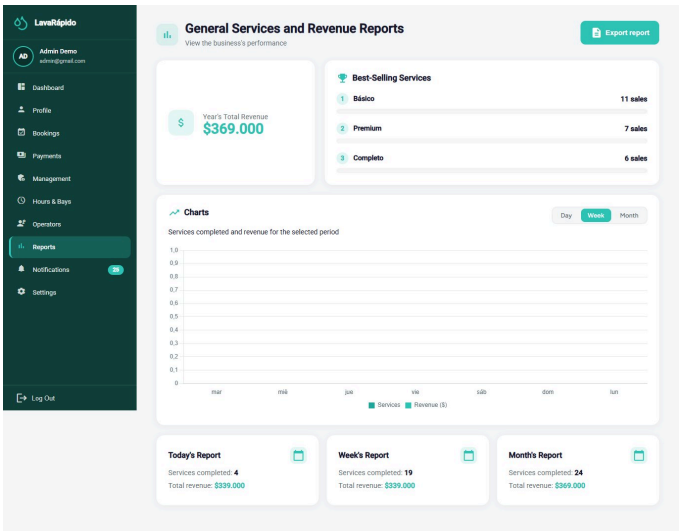
Screen	Evidence
Modal: Add service	
Management → Promotions	

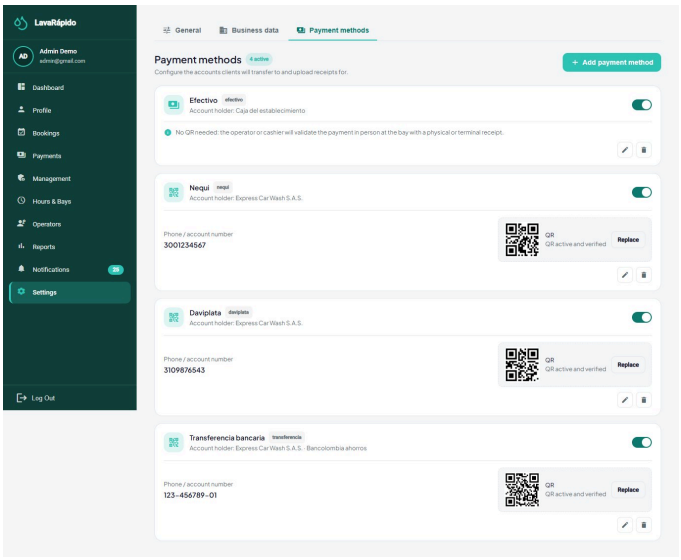
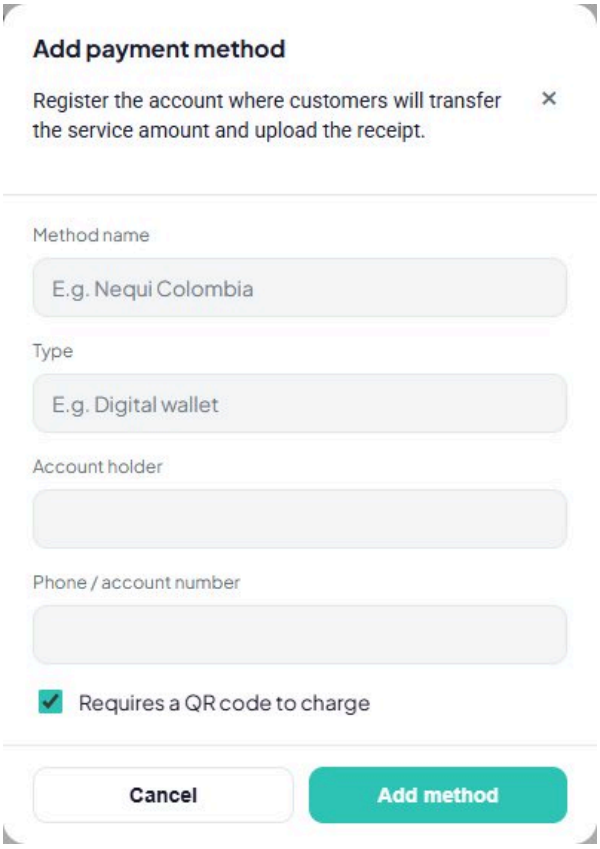
Screen	Evidence
Modal: Create promotion	
Opening hours and bays	

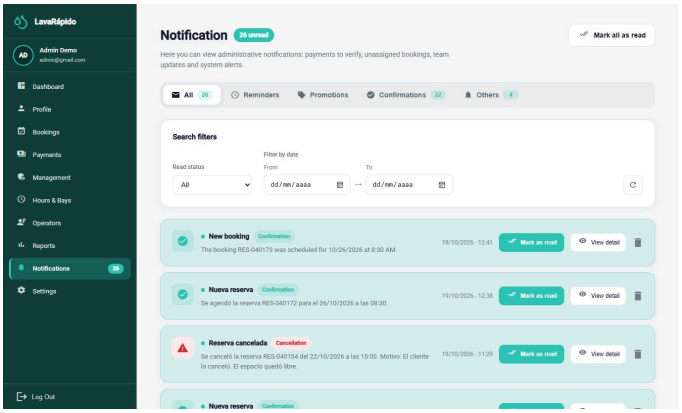
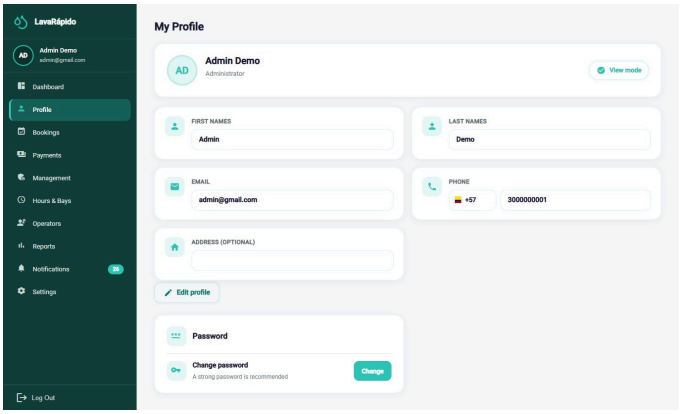
Screen	Evidence
Modal: Add opening-hours exception	
Operators	

Screen	Evidence
Modal: New operator	
Modal: Assign shift	

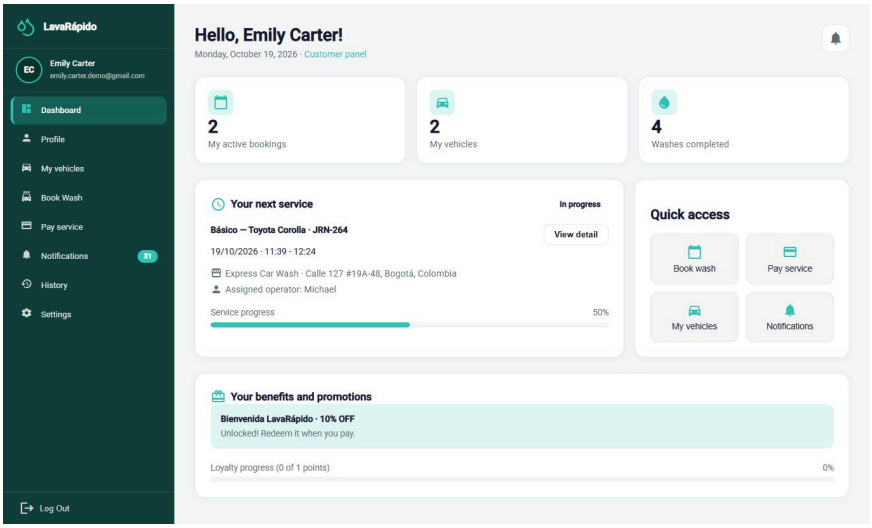
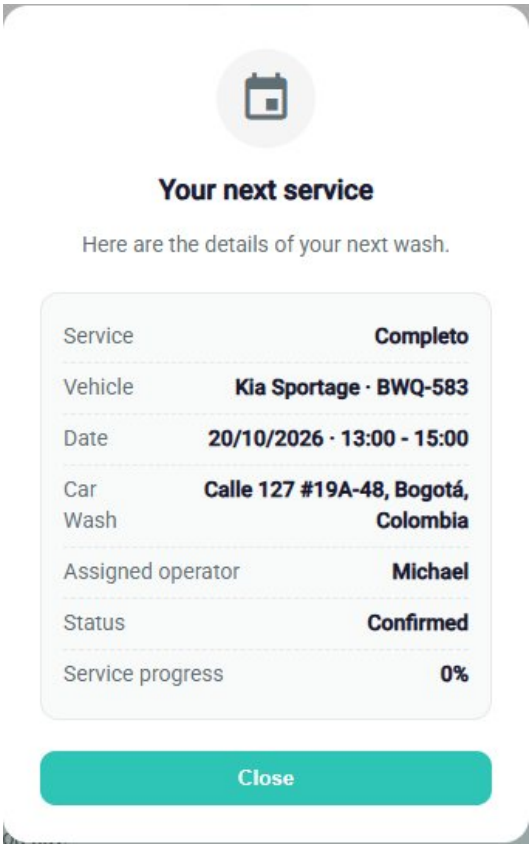
Screen	Evidence
Bookings	
Modal: Assign an operator to a booking	

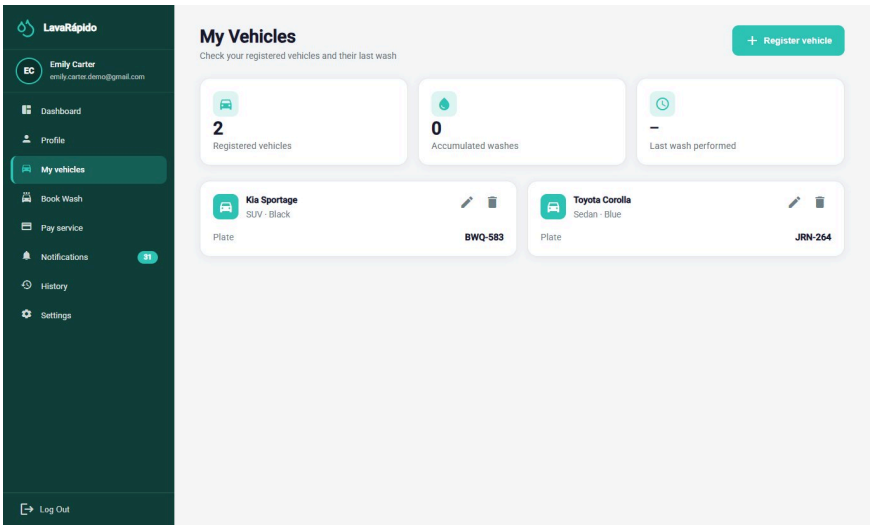
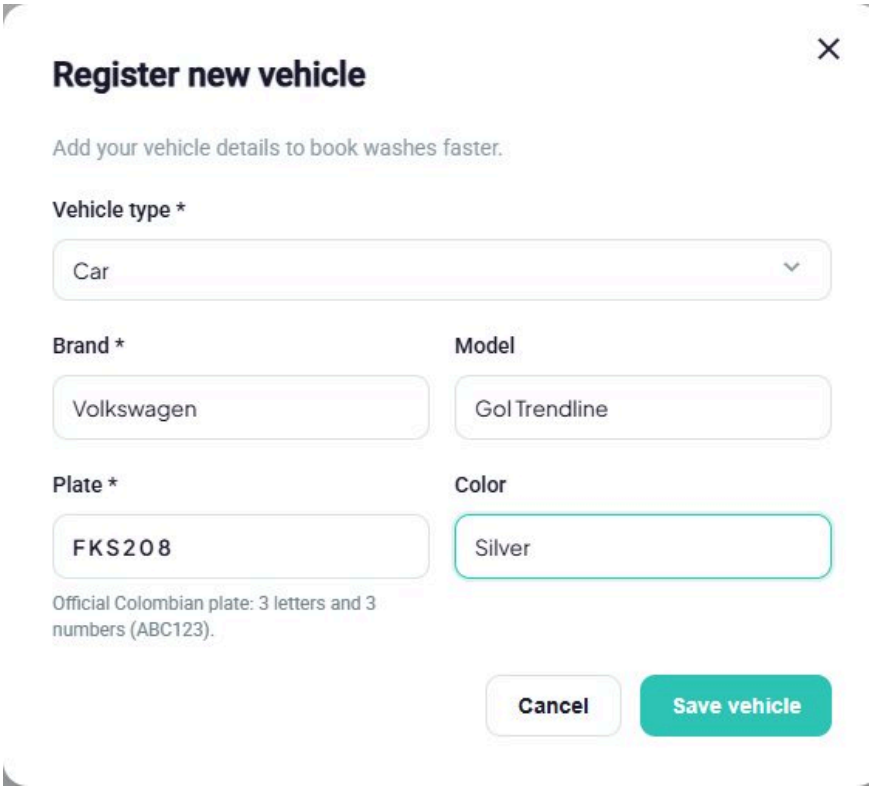
Screen	Evidence
Payments	
Modal: Record manual payment	
Reports	

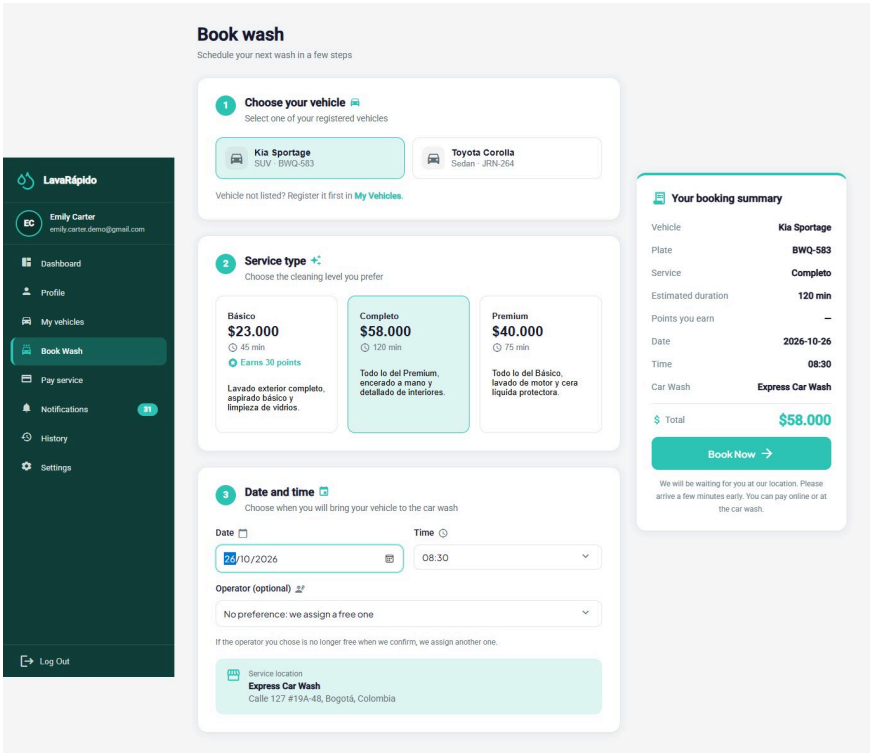
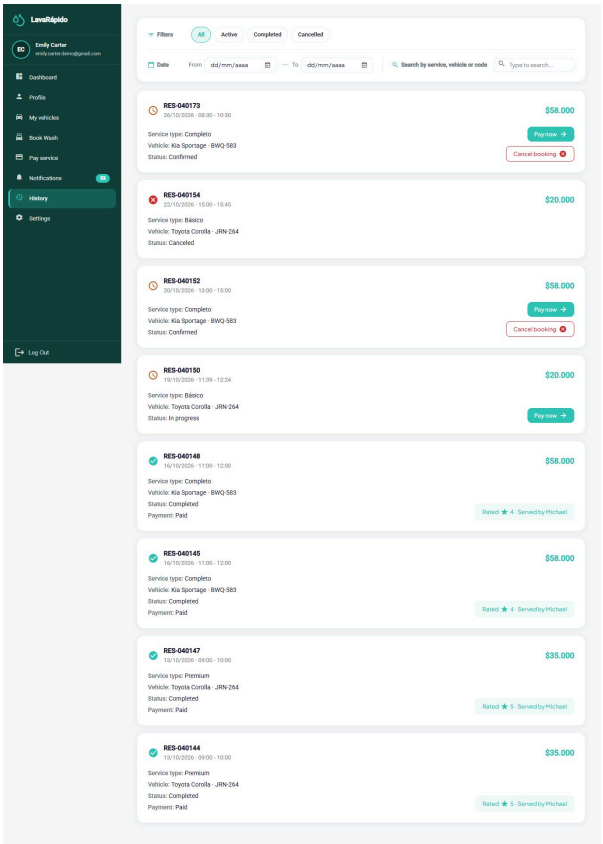
Screen	Evidence
Settings → Payment accounts	
Modal: Add payment account	

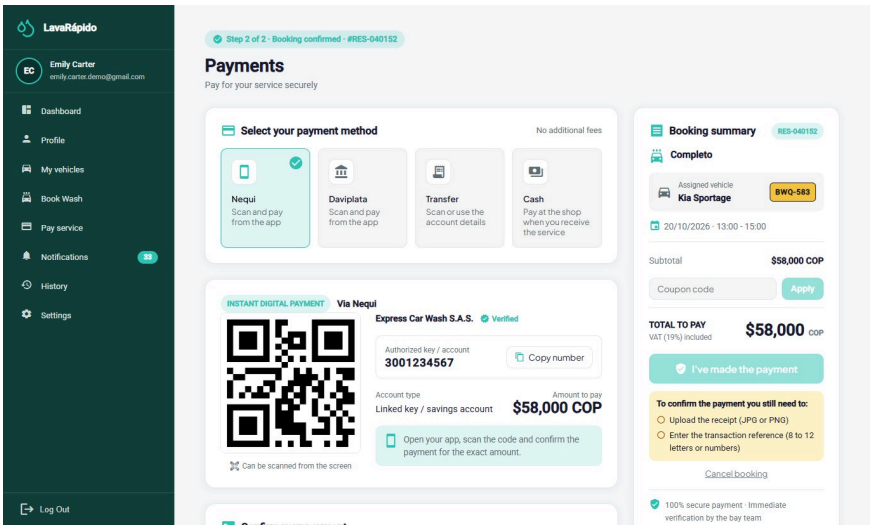
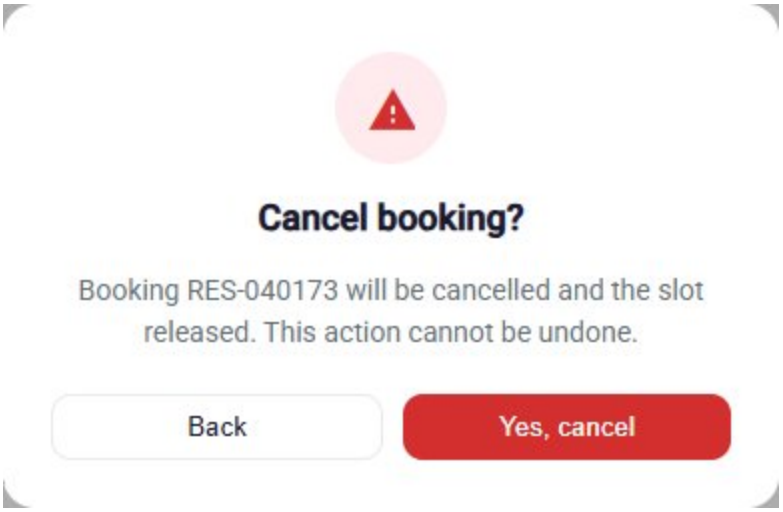
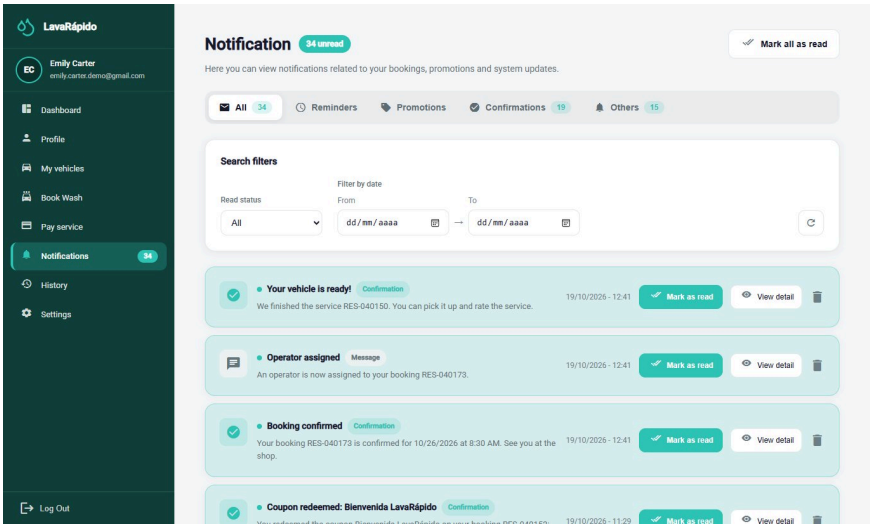
Screen	Evidence
Notifications	
Profile	

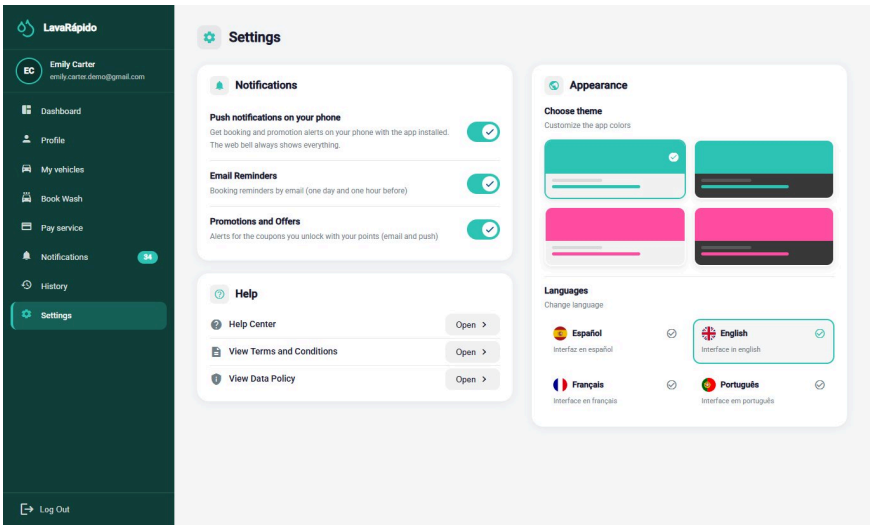
5.3 Customer Role

Screen	Evidence
Dashboard	
Modal: View details of the next service	

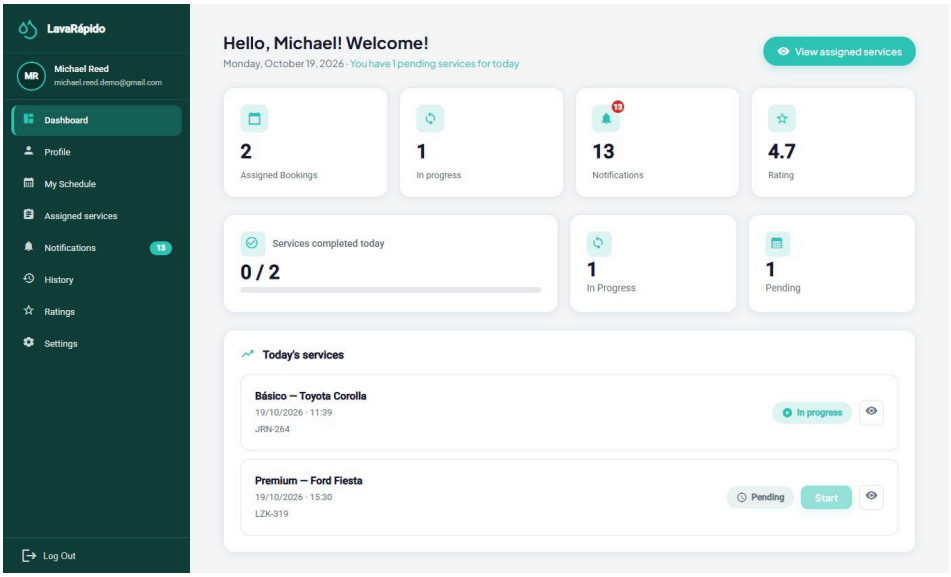
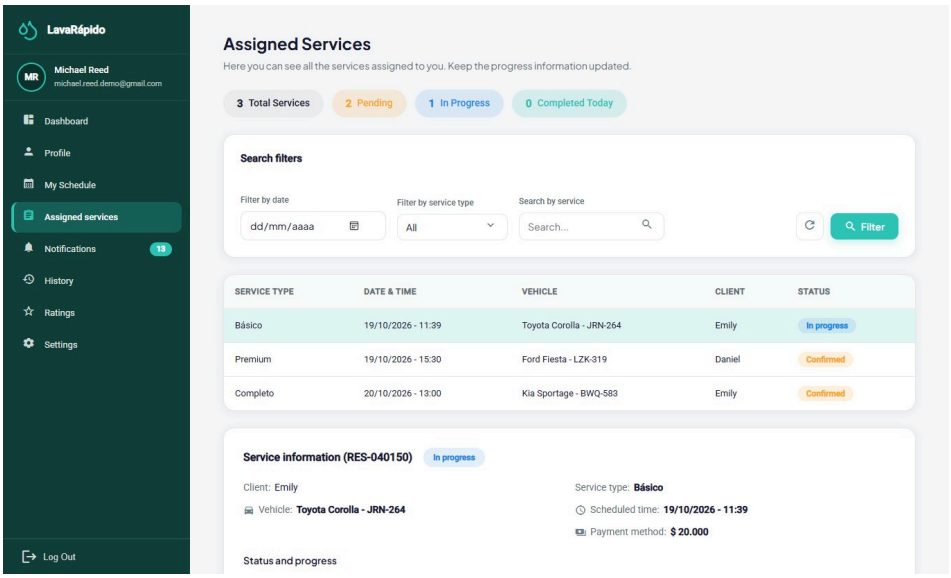
Screen	Evidence
My vehicles	
Modal: Register vehicle	

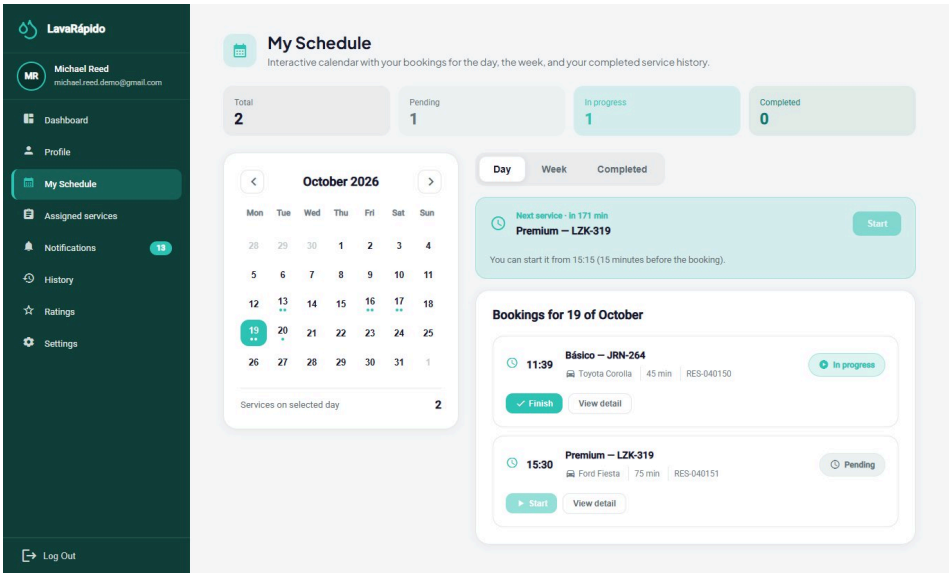
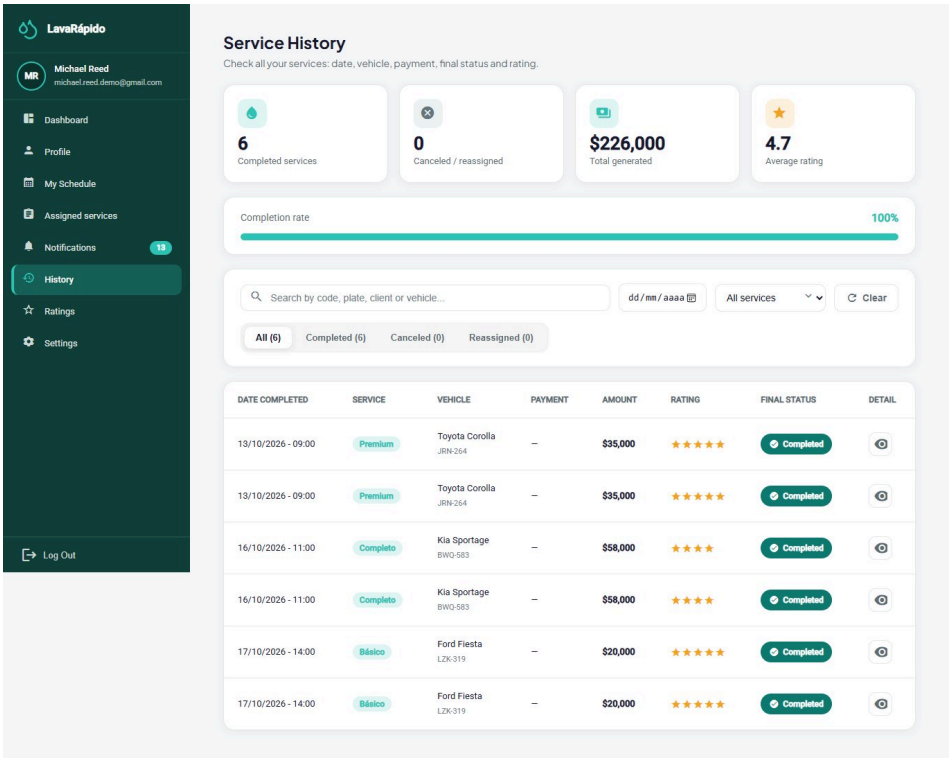
Screen	Evidence
Book a wash	
Booking history	

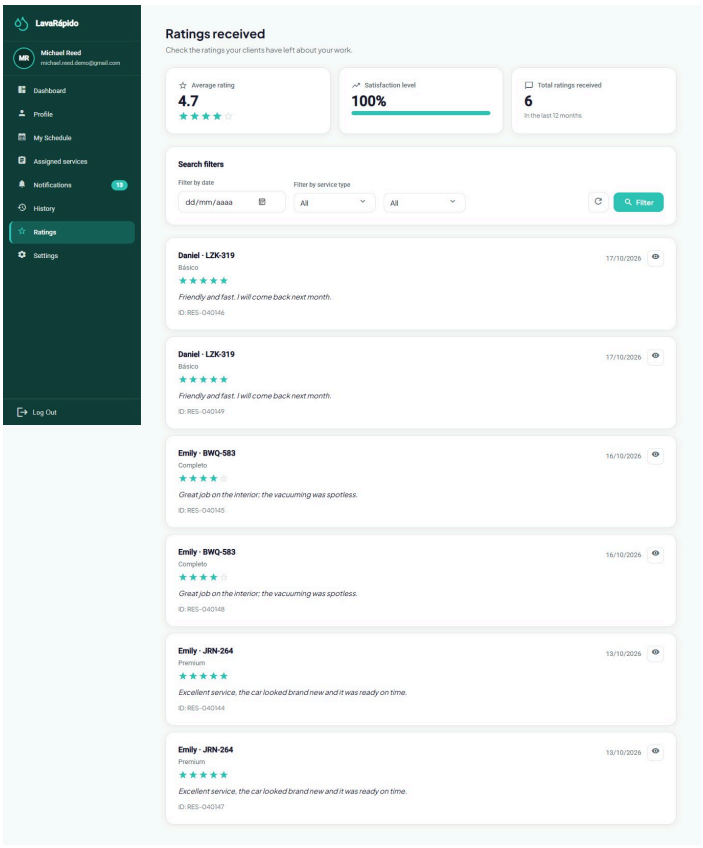
Screen	Evidence
Pay for a service	
Modal: Confirm booking cancellation	
Notifications	

Screen	Evidence
Settings (theme and language)	

5.4 Operator Role

Screen	Evidence
Main dashboard	
Assigned services	

Screen	Evidence
My schedule (availability)	
Service history	

Screen	Evidence
Ratings received	

Correlation With the Software Requirements Specification

Dynamic view in this document	Requirements it illustrates
§2.1 Registration and welcome	RF-001, RF-018
§2.2 Booking a wash	RF-005, RF-006, RF-010
§2.3 Reminders and alert	RF-018
§2.4 Assignment and execution	RF-008, RF-009, RF-010, RF-011, RF-012, RF-012b
§2.5 Payment by receipt	RF-013, RF-013b, RF-014, RF-016
§2.6 Coupon redemption and points	RF-024, RF-025, RF-026
§2.7 Roles and permissions	RF-029
§2.8 Sign-in with lockout	RF-002, RF-003, RF-004b
§2.9 Inspection report	RF-036, RF-018
§2.11 Expiry of the digital payment	RF-005, RF-033
§3.1 to §3.6 Activities	The whole authentication, booking, operations, payment and promotion module
§4.1 to §4.5 States	RF-006, RF-007, RF-012b, RF-013b, RF-014, RF-024, RF-036